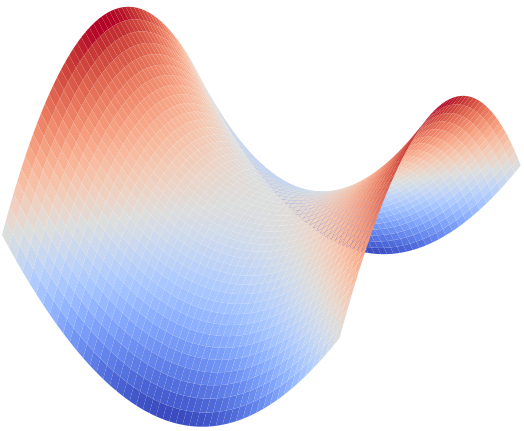
深度学习入门教程

Introduction to Deep Learning



偷书的孔乙己 著

VERSION: v0.9 | DATE: 2026 年 10 月 18 日

关于本书

本书《深度学习入门教程》（Introduction to Deep Learning）目前仍处于**撰写阶段**。由于本人平时有本职工作，所以只能利用业余时间推进，目前编写进度大概每月 3 到 4 个章节。

本书的整体知识脉络为：**神经网络基础知识** ⟶ **卷积神经网络** ⟶ **自然语言处理** ⟶ **大语言模型**。后续将视情况逐步补充 ViT、GAN、Diffusion 等相关主题。

本书文稿将同步发布于小红书、知乎与公众号（账号同名：偷书的孔乙己）。与此同时，全书的 LaTeX 源码会同步开源至 GitHub；因整理开源不易，插图会附带水印，希望大家理解与包涵。相关的代码实现目前计划等全书完结后再统一整理，之后视情况也可能会录制配套的教学讲解与代码实践视频。

本资料目前仅供个人学习与交流使用，未经许可请勿用于任何商业用途。感谢您的理解与支持！

本书处于撰写阶段 · 内容持续更新中

版权所有 © 2026 偷书的孔乙己

前言

在正式开始本书之前，我们需要先回答一个问题：**什么是深度学习？**

在经典的机器学习方法中，计算机完成一个具体任务，通常需要人为设计特征。以图像分类为例，研究者需要根据经验提取边缘、纹理、形状等信息，再将这些特征输入分类器进行学习；对于文本处理，也需要预先构造词袋、词频等特征表示。这种方法在任务结构相对明确、数据规模有限的场景下具有较好的可解释性，但其性能高度依赖人工设计的特征质量。而随着数据规模不断扩大、数据形式日益复杂，人工设计特征逐渐成为机器学习中的瓶颈。

深度学习（Deep Learning）的核心思想，是利用具有多层结构的人工神经网络，让模型直接从数据中学习有效的特征表示。传统机器学习需要人工设计特征，再利用这些特征完成分类、预测等任务；而深度学习则可以通过端到端的训练，使模型在完成具体任务的同时自动学习适合当前任务的特征表示。训练过程中，模型利用反向传播和梯度下降不断调整网络参数，使模型逐步学习到更有效的特征表示。

深度学习中的“深度”二字，主要体现在网络层次的逐级堆叠。对于图像数据，网络可以从较低层次的边缘、纹理等局部模式开始，逐步组合出更复杂的形状、部件以及完整目标；对于文本数据，模型则可以从词或子词等基础表示出发，逐步学习上下文关系、句法结构以及更高层次的语义信息。因此，深度学习不仅是一类具体的模型结构，也是一种重要的**表征学习（Representation Learning）**范式。

随着计算能力、数据规模以及优化算法的发展，深度学习已广泛应用于计算机视觉、自然语言处理、语音识别和推荐系统等领域。近年来，以卷积神经网络、Transformer 等为代表的深度学习模型不断拓展应用范围，并推动了图像理解、序列建模以及语言生成等任务的发展。

本书默认读者具备以下基本数学基础：

- **微积分：**掌握导数、偏导数、链式法则、梯度以及雅可比矩阵等基本概念；
- **线性代数：**掌握向量和矩阵的基本运算，以及行列式、秩、特征值与特征向量等基本概念；
- **概率论与数理统计：**掌握条件概率、常见概率分布、期望、方差、协方差以及极大似然估计等基本概念。

以上数学知识以能够理解和完成书中的相关推导为基本要求，不涉及过于深入或抽象的数学理论。对于书中涉及公式推导的部分，建议读者结合具体计算过程亲自动手演算，以加深对相关概念和计算过程的理解，而不是仅仅记忆结论。

共勉。

偷书的孔乙己

2026 年于天津

目录

前言 i

I 神经网络入门 1

第 1 章 多层感知机 3

1.1 神经元与感知机模型 3

1.2 感知机模型的局限 5

1.3 多层感知机模型 5

1.4 多层感知机的矩阵表示 9

1.5 参数设置的局限 10

第 2 章 反向传播算法 12

2.1 损失函数 12

2.2 激活函数 14

2.3 反向传播算法 16

2.4 梯度下降算法 20

2.5 计算图与自动微分 22

2.6 万能逼近定理 25

第 3 章 神经网络优化算法 27

3.1 学习率 28

3.2 批量策略 32

3.3 动量法 34

3.4 自适应学习率算法 36

3.4.1 AdaGrad 36

3.4.2 RMSProp 38

3.4.3 Adam 39

第 I 部分
神经网络入门

第 1 章　多层感知机

1.1　神经元与感知机模型

在生物神经系统中，神经元（Neuron）是信息处理的基本单元。早在 20 世纪 40 年代，神经科学家就发现，大脑中的神经元通过树突（Dendrites）接收来自其他神经元或感知器（如视网膜）的信号。这些信号通过突触（Synapse）传递，并携带一定的突触强度（也称突触权重），以调节信号对神经元激活程度的影响。多个输入在细胞体膜上整合后，如果累积输入超过激发阈值，神经元便会在轴丘（Axon Hillock）触发动作电位，并沿轴突（Axon）传输至下一个神经元或目标组织。

我们可以通过图 1.1来直观地观察神经元的典型结构：它由树突、细胞体（Soma）及其内部的细胞核（Nucleus）、轴突以及轴突末端（Axon Terminals）构成。

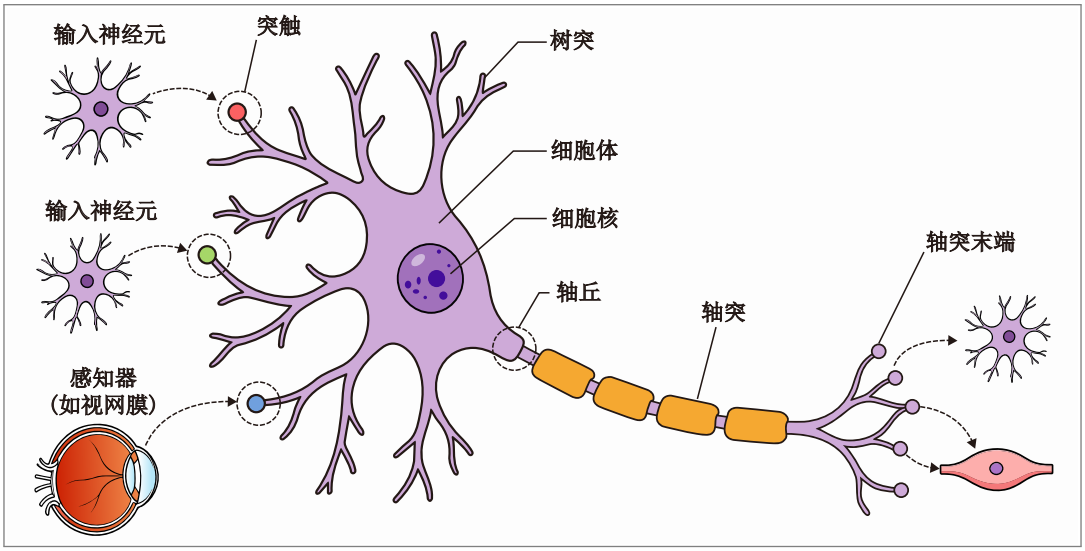


图 1.1　生物神经元结构示意图。图左侧的输入神经元与感知器通过突触将信号传递给中间神经元的树突；树突接收信号后将其送入细胞体进行整合。当累积输入超过激发阈值，则在轴丘处触发动作电位，并沿轴突传导至轴突末端，最终再经突触作用于下游神经元或目标组织。

受此启发，我们可以把这套生物机制抽象成一个简洁的数学模型。首先，模型接收若干输入信号 $x_1, x_2, x_3, \ldots$，每个输入信号都有对应的权重 $w_1, w_2, w_3, \ldots$。将信号与权重相乘，就能表示该信号对最终决策的贡献度。随后，将所有加权的输入进行求和，并加上一个偏置项 b（Bias），就得到了神经元的总输入信号 z。这个过程可以表示为：

$$z = \sum_i w_i x_i + b$$

接下来，我们需要一个机制来判断这个总输入 z 是否达到了触发条件。最直接的做法是

设定一个阈值：若超过阈值，神经元激活（输出 1）；否则保持沉寂（输出 0）。基于这一思想，我们便重新发明了人工神经网络中最早的模型——**感知机（Perceptron）**[1]。它的完整数学表达式为：

$$y = f(z) = f(\sum_i w_i x_i + b)$$

在这里，$f(x)$ 被称为**激活函数**。在感知机模型中，它是一个阶跃函数（Step Function），用于判断总输入信号是否超过了某个预先设定的阈值，从而确定神经元是否被激活：

$$f(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

为了更直观地理解，我们可以将感知机模型应用于一个简单的分类任务：判断一位学生能否通过某门课程。输入为平时成绩 $x_1 \in [0, 100]$ 和期末成绩 $x_2 \in [0, 100]$，对应的权重分别为 w_1 和 w_2，权重的大小表示这两项成绩在最终评估中的占比。假设平时成绩占总评的 40%，期末成绩占 60%（即 $w_1 = 0.4, w_2 = 0.6$），且不考虑额外的加分或扣分，即偏置项设定为 0，总分达到或超过 60 分即为通过。顺着这个逻辑，我们可以构造出一个明确的感知机分类器：

$$f(x_1, x_2) = \begin{cases} \text{通过}, & \text{if } 0.4x_1 + 0.6x_2 \geq 60 \\ \text{挂科}, & \text{otherwise} \end{cases}$$

通过这一模型，我们可以轻松计算出某位学生的最低达标要求。例如，如果一位学生的平时成绩为 $x_1 = 30$ 分，代入公式后，其期末成绩 x_2 最低需满足 $0.4 \times 30 + 0.6x_2 \geq 60$，方可通过。解得 $x_2 \geq 80$。该分类问题的分界线如图 1.2所示：

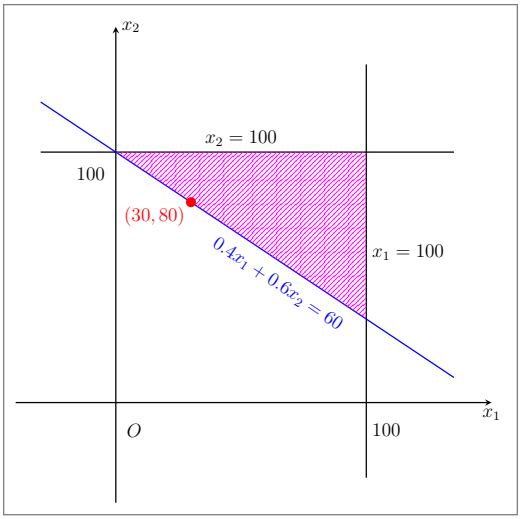


图 1.2 感知机分类决策边界示意图。以平时成绩 x_1 和期末成绩 x_2 为坐标轴，蓝色斜线 $0.4x_1 + 0.6x_2 = 60$ 为决策边界，其上方阴影区域（含边界）为通过区域，其余为挂科；红点 $(30, 80)$ 落在边界上，对应平时 30 分需期末至少 80 分才能通过的临界情形。

[1] 感知机模型由美国心理学家 Frank Rosenblatt 于 1957 年提出，原始论文参见 *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*。该模型首次引入了基于误差驱动的权重更新算法，使机器具备了从数据中学习的能力。这一突破在当时的人工智能领域引发了巨大轰动，学界一度乐观地认为，机器智能的实现已近在咫尺。

1.2 感知机模型的局限

从刚刚的例子可以看出，感知机模型简单、直观且计算高效。然而，让我们重新审视感知机的核心判决逻辑（以二维数据为例）：

$$f(x_1, x_2) = \begin{cases} 1, & \text{if } w_1 x_1 + w_2 x_2 + b > 0 \\ 0, & \text{otherwise} \end{cases}$$

可以看出，感知机的分类边界在二维空间中，永远是一条笔直的线（$w_1 x_1 + w_2 x_2 + b = 0$）。在三维空间中，它是一个平整的面；在更高维空间中，则是一个超平面（即高维空间中的线性分类边界，可以简单理解为高维空间中的“平面”）。这意味着它只能解决**线性可分（Linearly Separable）**的问题。如果现实世界的数据分布稍微复杂一点，无法被一刀切开，感知机便会暴露出其致命的软肋。

我们可以考虑一个具体的例子：如图 1.3所示，在二维平面中，点 $(0,0)$、$(1,1)$ 属于同一类别，而点 $(0,1)$、$(1,0)$ 属于另一类别。可以发现，两类样本并不是简单地分布在某条直线的两侧，而是呈现出交叉对立的分布形式，这就是逻辑运算中著名的异或（XOR）问题。

对于这样的数据分布，我们可以拿尺子在图上任意比划一下，就会发现：无论怎样调整直线的位置和方向，都不可能找到一条直线，将这两类点完美分隔开来[2]。

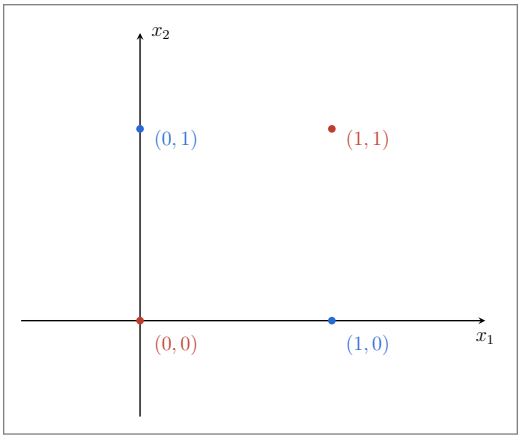


图 1.3 异或（XOR）问题示意图。四个样本点分为两类：$(0,0)$、$(1,1)$（红色）输出为 0，$(0,1)$、$(1,0)$（蓝色）输出为 1。两类点各占对角、交叉分布。

1.3 多层感知机模型

感知机模型的设计本身受到了生物神经系统的启发。在生物神经系统中，神经元并不是孤立工作的，而是通过大量神经元之间的连接形成多层次的网络，共同完成信息的传递与处理。那既然单个感知机的表达能力有限，那么能否将多个感知机按照一定的层级结构进行组合，从而获得更强的表示能力，并进一步解决异或这样的非线性问题呢？基于这一思路，我们

[2] 1969 年，Marvin Minsky 和 Seymour Papert 出版了《感知机》（*Perceptrons*）一书。该书通过数学分析指出，单层感知机无法解决异或（XOR）等非线性可分问题。

可以构建一个简单的**多层感知机模型**，其具体结构如图 1.4所示：

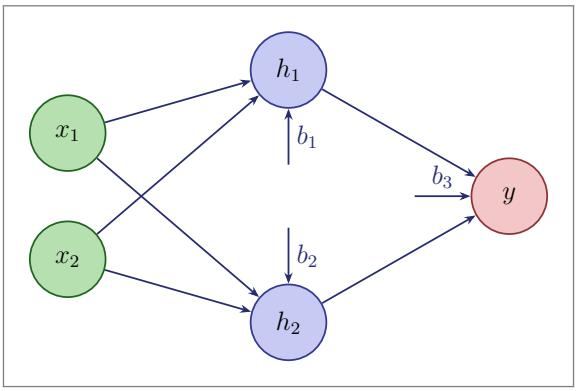


图 1.4 没有激活函数的多层感知机模型。输入层 x_1, x_2、隐藏层 h_1, h_2 与输出层 y 逐层全连接，偏置 b_1, b_2, b_3 注入对应神经元；各节点仅做线性加权求和。

如图所示，该模型增加了两个中间节点：两个输入节点 x_1 和 x_2 的信号不再直接决定结果，而是先传递给中间节点 h_1 和 h_2。这些位于输入层和输出层之间的节点被称为**隐藏层 (Hidden Layer)**，它们负责对输入信息进行进一步的变换与组合，随后将处理后的信息传递给输出节点 y，由输出节点得到最终结果。假设暂时不引入激活函数，仅依靠神经元内部的线性加权求和（如 $z_1 = w_{11}x_1 + w_{12}x_2 + b_1$），则整个网络的计算过程可表示为：

$$
\begin{aligned}
h_1 &= w_{11}x_1 + w_{12}x_2 + b_1 \\
h_2 &= w_{21}x_1 + w_{22}x_2 + b_2 \\
y &= v_1h_1 + v_2h_2 + b_3 \\
&= v_1(w_{11}x_1 + w_{12}x_2 + b_1) + v_2(w_{21}x_1 + w_{22}x_2 + b_2) + b_3 \\
&= v_1w_{11}x_1 + v_1w_{12}x_2 + v_1b_1 + v_2w_{21}x_1 + v_2w_{22}x_2 + v_2b_2 + b_3 \\
&= (v_1w_{11} + v_2w_{21})x_1 + (v_1w_{12} + v_2w_{22})x_2 + (v_1b_1 + v_2b_2 + b_3)
\end{aligned}
$$

稍加整理后我们可以发现，若定义新权重 $W_1 = v_1w_{11} + v_2w_{21}$、$W_2 = v_1w_{12} + v_2w_{22}$，以及新偏置 $B = v_1b_1 + v_2b_2 + b_3$，那整个模型就可以写成：

$$y = W_1x_1 + W_2x_2 + B$$

可以看到，虽然我们在网络中增加了隐藏层，将多个神经元进行了堆叠，但在没有激活函数的情况下，这些线性计算始终可以合并成一次线性加权求和。换句话说，在没有激活函数的情况下，无论网络堆叠多少层，只要每一层都只是线性变换，整个网络本质上仍然只是一个线性模型，并不会因为增加网络层数而获得更强的表达能力。

因此，单纯依靠线性层的堆叠并不能突破感知机的局限。既然如此，我们不妨加入之前介绍过的激活函数，看看情况是否会有所不同。如图 1.5所示，输入信号 x_1 和 x_2 首先分别经过加权求和并加上偏置，得到两个中间结果；随后，这两个结果分别经过激活函数，得到 h_1 和 h_2。接着，h_1 和 h_2 再进行加权求和并加上偏置，最后经过激活函数得到输出 y。其完整

的数学公式也随之变为：

$$
\begin{aligned}
h_1 &= f(w_{11}x_1 + w_{12}x_2 + b_1) \\
h_2 &= f(w_{21}x_1 + w_{22}x_2 + b_2) \\
y &= f(v_1h_1 + v_2h_2 + b_3)
\end{aligned}
$$

其中，$f(x)$ 表示阶跃函数。

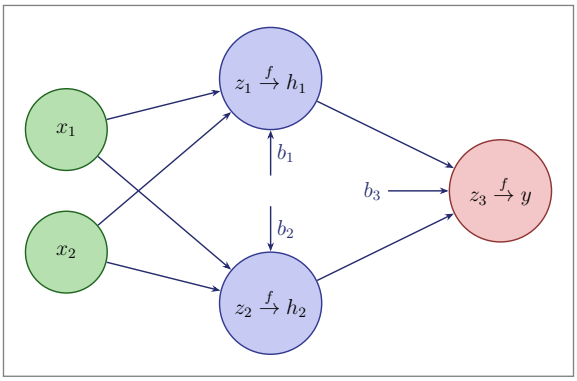


图 1.5 带有激活函数的多层感知机模型。结构与图 1.4相同，但在隐藏层和输出层的线性加权求和之后加入阶跃函数 f，分别得到 h_1、h_2 和最终输出 y。

为验证该结构能否解决异或问题，我们设定以下具体的权重与偏置参数：

$$
\begin{aligned}
&w_{11} = 1, \quad w_{12} = 1, \quad b_1 = -0.5 \\
&w_{21} = 1, \quad w_{22} = 1, \quad b_2 = -1.5 \\
&v_1 = 1, \quad v_2 = -2, \quad b_3 = -0.5 \\
&\downarrow \\
&h_1 = f(x_1 + x_2 - 0.5) \\
&h_2 = f(x_1 + x_2 - 1.5) \\
&y = f(h_1 - 2h_2 - 0.5)
\end{aligned}
$$

情形一：输入为 $(x_1, x_2) = (0, 0)$

$$
\begin{aligned}
z_1 &= 1 \cdot 0 + 1 \cdot 0 - 0.5 = -0.5 \quad \Rightarrow \quad h_1 = f(-0.5) = 0 \\
z_2 &= 1 \cdot 0 + 1 \cdot 0 - 1.5 = -1.5 \quad \Rightarrow \quad h_2 = f(-1.5) = 0 \\
z_3 &= 1 \cdot 0 - 2 \cdot 0 - 0.5 = -0.5 \quad \Rightarrow \quad y = f(-0.5) = 0
\end{aligned}
$$

情形二：输入为 $(x_1, x_2) = (1, 1)$

$$
\begin{aligned}
z_1 &= 1 \cdot 1 + 1 \cdot 1 - 0.5 = 1.5 \quad \Rightarrow \quad h_1 = f(1.5) = 1 \\
z_2 &= 1 \cdot 1 + 1 \cdot 1 - 1.5 = 0.5 \quad \Rightarrow \quad h_2 = f(0.5) = 1 \\
z_3 &= 1 \cdot 1 - 2 \cdot 1 - 0.5 = -1.5 \quad \Rightarrow \quad y = f(-1.5) = 0
\end{aligned}
$$

情形三：输入为 $(x_1, x_2) = (0, 1)$

$$z_1 = 1 \cdot 0 + 1 \cdot 1 - 0.5 = 0.5 \quad \Rightarrow \quad h_1 = f(0.5) = 1$$
$$z_2 = 1 \cdot 0 + 1 \cdot 1 - 1.5 = -0.5 \quad \Rightarrow \quad h_2 = f(-0.5) = 0$$
$$z_3 = 1 \cdot 1 - 2 \cdot 0 - 0.5 = 0.5 \quad \Rightarrow \quad y = f(0.5) = 1$$

情形四：输入为 $(x_1, x_2) = (1, 0)$

$$z_1 = 1 \cdot 1 + 1 \cdot 0 - 0.5 = 0.5 \quad \Rightarrow \quad h_1 = f(0.5) = 1$$
$$z_2 = 1 \cdot 1 + 1 \cdot 0 - 1.5 = -0.5 \quad \Rightarrow \quad h_2 = f(-0.5) = 0$$
$$z_3 = 1 \cdot 1 - 2 \cdot 0 - 0.5 = 0.5 \quad \Rightarrow \quad y = f(0.5) = 1$$

汇总上述计算结果，可以得到表 1.1。可以看到，模型对 4 组输入均做出了正确判断，说明通过引入隐藏层和激活函数，多层感知机已经成功解决了异或问题。

表 1.1 异或问题输入输出表

输入 x_1	输入 x_2	模型输出 y	期望输出	是否正确
0	0	0	0	正确
1	1	0	0	正确
0	1	1	1	正确
1	0	1	1	正确

通过上述内容可以发现，非线性激活函数的主要作用是为模型引入非线性，使模型能够学习和表示更加复杂的非线性关系。

深度学习名称的由来

“深度学习”（Deep Learning）从网络结构上看，本质就是具有多层隐藏层的神经网络。但在第二次人工智能寒冬期间（20 世纪末至 21 世纪初），“神经网络”一词在学术界几乎声名狼藉，学术评审对神经网络方向存在强烈的偏见。带“Neural Network”关键词的稿件极易在初审阶段被直接驳回，相关的科研经费也难以获批。

转机出现在 2006 年。Geoffrey Hinton、Simon Osindero 和 Yee-Whye Teh 在论文*A Fast Learning Algorithm for Deep Belief Nets*中提出基于逐层无监督预训练的深度信念网络方法，为深层网络的训练提供了新的思路。同年，Geoffrey Hinton 与 Ruslan Salakhutdinov 在《Science》发表论文Reducing the Dimensionality of Data with Neural Networks，进一步验证了深层网络在降维与特征学习上的潜力。Hinton 等人策略性地引入并推广了“深度信念网络”与“深度学习”等新概念，以规避当时对“神经网络”的严重偏见。此后几年，“深度学习”这一统称逐渐在学术界流传开来。

2012 年，AlexNet 在 ImageNet 竞赛中以显著优势夺冠，深度卷积网络展现出的强大性能成为深度学习复兴的重要转折点。此后，随着计算能力和数据规模的快速增长，“深度学习”逐渐成为多层网络研究的主流，并发展为今天人工智能领域的核心技术体系。

1.4 多层感知机的矩阵表示

前面我们通过具体的权重和偏置，逐个计算了多层感知机中的每个神经元。虽然这种写法能够直观展示网络的计算过程，但当网络层数和神经元数量增加后，逐个书写每个神经元的计算公式会变得越来越繁琐。实际上，我们可以把同一层中的神经元统一表示成向量，并进一步使用矩阵来描述整个网络的计算过程。如图 1.5所示，输入层与隐藏层之间、隐藏层与输出层之间的神经元均相互连接，因此这种网络也可以称为**全连接网络（Fully Connected Network）**。对于全连接层，每个输出神经元都会接收上一层所有神经元的输入，并为每一条连接学习一个独立的权重。

以当前的多层感知机为例，我们可以将输入 x_1, x_2 组成输入向量：

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

隐藏层包含两个神经元，因此也可以将它们的线性输出写成向量：

$$\boldsymbol{z}^{(1)} = \begin{pmatrix} z_1 \\ z_2 \end{pmatrix}$$

而对应的权重和偏置可分别写成：

$$W^{(1)} = \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix} \qquad \boldsymbol{b}^{(1)} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$$

这样，隐藏层原来分别写成：

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \qquad z_2 = w_{21}x_1 + w_{22}x_2 + b_2$$

的两个标量计算，就可以统一写成一个矩阵运算：

$$\boldsymbol{z}^{(1)} = W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}$$

如果隐藏层使用激活函数 f，则有：

$$\boldsymbol{h} = f\left(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\right)$$

其中 f 作用于向量中的每一个元素。

同样，对于输出层，将隐藏层的两个输出组成向量 $\boldsymbol{h}$，输出层的权重可以写成：

$$W^{(2)} = \begin{pmatrix} v_1 & v_2 \end{pmatrix} \qquad b^{(2)} = b_3$$

因此，输出层的计算可以写成：

$$z^{(2)} = W^{(2)}\boldsymbol{h} + b^{(2)}$$

经过激活函数后得到最终输出：

$$y = f\left(W^{(2)}\boldsymbol{h} + b^{(2)}\right)$$

将隐藏层代入其中，整个多层感知机就可以进一步写成：

$$y = f\left(W^{(2)} f\left(W^{(1)}\boldsymbol{x} + \boldsymbol{b}^{(1)}\right) + b^{(2)}\right)$$

这样一来，原来需要分别计算每个神经元的多组公式，就被统一成了矩阵乘法和向量加法。更一般地，对于一个全连接层，如果上一层的输出包含 N_{in} 个神经元，当前层包含 N_{out} 个神经元，那么可以将上一层的输出表示为向量 $\boldsymbol{x} \in \mathbb{R}^{N_{\text{in}}}$，权重表示为 $W \in \mathbb{R}^{N_{\text{out}} \times N_{\text{in}}}$，偏置表示为 $\boldsymbol{b} \in \mathbb{R}^{N_{\text{out}}}$。整个全连接层就可以统一写成：

$$\boldsymbol{z} = W\boldsymbol{x} + \boldsymbol{b}$$

再经过激活函数得到：

$$\boldsymbol{y} = f(\boldsymbol{z})$$

其中，权重矩阵 W 的第 i 行对应第 i 个输出神经元与上一层所有输入之间的连接权重。因此，矩阵乘法 $W\boldsymbol{x}$ 实际上就是同时完成了这一层所有神经元的加权求和。这种矩阵表示不仅让网络的计算形式更加简洁，也为后续的反向传播提供了统一的数学基础。

1.5 参数设置的局限

在刚才的验证中，我们通过一组精心设计的权重和偏置，使多层感知机成功解决了异或问题。然而，这也带来了一个新的问题：**如果这些参数不是人为设计好的，而是随意设置的一组初始值，模型还能正常完成分类吗？**

为了直观地观察这一点，我们对图 1.5中的模型进行进一步测试。我们不再针对异或问题刻意调整参数，而是直接将所有权重设为 1，所有偏置设为 0：

$$\begin{aligned} w_{11} = 1, \quad w_{12} = 1, \quad b_1 = 0 \\ w_{21} = 1, \quad w_{22} = 1, \quad b_2 = 0 \\ v_1 = 1, \quad v_2 = 1, \quad b_3 = 0 \end{aligned}$$

现在，我们将异或问题的四组输入依次代入该模型，并使用阶跃函数作为激活函数，观察模型的输出结果。

情形一：输入为 $(x_1, x_2) = (0, 0)$

$$\begin{aligned} z_1 = 1 \cdot 0 + 1 \cdot 0 + 0 = 0 \quad &\Rightarrow \quad h_1 = f(0) = 0 \\ z_2 = 1 \cdot 0 + 1 \cdot 0 + 0 = 0 \quad &\Rightarrow \quad h_2 = f(0) = 0 \\ z_3 = 1 \cdot 0 + 1 \cdot 0 + 0 = 0 \quad &\Rightarrow \quad y = f(0) = 0 \end{aligned}$$

情形二：输入为 $(x_1, x_2) = (1, 1)$

$$\begin{aligned} z_1 = 1 \cdot 1 + 1 \cdot 1 + 0 = 2 \quad &\Rightarrow \quad h_1 = f(2) = 1 \\ z_2 = 1 \cdot 1 + 1 \cdot 1 + 0 = 2 \quad &\Rightarrow \quad h_2 = f(2) = 1 \\ z_3 = 1 \cdot 1 + 1 \cdot 1 + 0 = 2 \quad &\Rightarrow \quad y = f(2) = 1 \end{aligned}$$

情形三：输入为 $(x_1, x_2) = (0, 1)$

$$z_1 = 1 \cdot 0 + 1 \cdot 1 + 0 = 1 \quad \Rightarrow \quad h_1 = f(1) = 1$$
$$z_2 = 1 \cdot 0 + 1 \cdot 1 + 0 = 1 \quad \Rightarrow \quad h_2 = f(1) = 1$$
$$z_3 = 1 \cdot 1 + 1 \cdot 1 + 0 = 2 \quad \Rightarrow \quad y = f(2) = 1$$

情形四：输入为 $(x_1, x_2) = (1, 0)$

$$z_1 = 1 \cdot 1 + 1 \cdot 0 + 0 = 1 \quad \Rightarrow \quad h_1 = f(1) = 1$$
$$z_2 = 1 \cdot 1 + 1 \cdot 0 + 0 = 1 \quad \Rightarrow \quad h_2 = f(1) = 1$$
$$z_3 = 1 \cdot 1 + 1 \cdot 1 + 0 = 2 \quad \Rightarrow \quad y = f(2) = 1$$

汇总结果如表 1.2所示：

表 1.2 错误参数下异或问题的分类结果

输入 x_1	**输入** x_2	**模型输出** y	**期望输出**	**是否正确**
0	0	0	0	正确
1	1	1	0	**错误**
0	1	1	1	正确
1	0	1	1	正确

可以看到，在这组随意设置的参数下，模型面对输入 $(1, 1)$ 时出现了错误预测，说明仅仅构建多层感知机并不能保证它能够正确解决异或问题。要使模型得到正确的结果，还需要为网络设置合适的权重和偏置。对于异或这样简单的问题，由于模型的参数量很少，我们可以通过手工计算和反复尝试找到一组合适的参数；但在实际任务中，模型往往包含成千上万，甚至数以亿计的参数，依靠人工逐一调整参数显然是不现实的。

> 那我们能不能设计一种方法，让模型根据预测结果与真实结果之间的误差，自动调整网络中的参数，使模型的预测逐渐变得更加准确？[3]

[3] Frank Rosenblatt 为早期感知机设计的参数更新规则主要适用于单层感知机，无法有效训练多层感知机中的隐藏层参数。虽然多层网络具有解决非线性问题的潜力，但当时缺乏有效的训练方法，这一局限导致神经网络研究陷入低潮，也是 20 世纪 70 年代第一次人工智能寒冬的重要原因之一。

第 2 章　反向传播算法

多层感知机虽然具备了表示复杂非线性关系的能力，但模型能否真正完成任务，最终取决于权重和偏置是否设置合理。在实际应用中，这些参数无法依靠人工逐一设定，而是需要模型在训练过程中根据预测结果不断进行调整。那么，模型应该如何判断自己的预测是否准确，又应该如何调整这些参数?

2.1　损失函数

首先，我们需要让模型知道自己的预测结果是否准确。对于模型给出的一个预测结果，我们需要将其与真实目标进行比较，从而判断模型的预测结果是否正确，以及预测结果与真实目标之间存在多大的差距。这个用于衡量预测结果与真实目标之间差距的数学函数，就是**损失函数（Loss Function）**。一般来说，损失函数的值越大，说明模型的预测结果与真实目标相差越远；损失值越小，则说明模型的预测结果越接近真实目标。

假设对于某一个输入样本 x，模型经过计算得到的预测结果为 y_{pred}，而真实标签为 y_{true}。要衡量二者之间的差距，最直接的做法，就是直接计算二者之间的差值：

$$\mathcal{L} = y_{\text{pred}} - y_{\text{true}}$$

这个差值可以表示模型预测方向上的偏差。当 $\mathcal{L} > 0$ 时，说明模型预测值偏大；当 $\mathcal{L} < 0$ 时，说明模型预测值偏小；当 $\mathcal{L} = 0$ 时，则表示模型预测完全正确。

然而，在实际任务中，我们通常面对的并不是一个样本，而是包含大量样本的数据集。为了衡量模型在整个数据集上的预测误差，就需要将各个样本产生的误差进行累加。如果直接使用预测值与真实值之间的差值，不同样本的正负误差就可能相互抵消。例如，模型在一个样本上的预测偏差为 $+5$，在另一个样本上的预测偏差为 -5，虽然二者方向相反，但实际误差大小都是 5。将它们直接相加后得到 $(+5) + (-5) = 0$，这就无法准确反映模型在这两个样本上的实际误差。

因此，我们需要消除误差的正负方向，只关注误差绝对值的大小。比较直接的做法，就是取绝对值。这样，无论原来的误差是正数还是负数，最终得到的结果都不会带有负号：

$$\mathcal{L} = |y_{\text{pred}} - y_{\text{true}}|$$

这就是**绝对误差（Absolute Error）**。它能直接表示预测值与真实值之间的距离，不会因为不同样本之间误差方向不同而相互抵消。

现在我们使用绝对误差作为损失函数，那么接下来的目标就是：让损失函数尽可能小，也就是寻找损失函数的最小值。需要注意的是，损失函数并不是一个固定不变的数值，而是由模型参数决定的函数。当模型中的参数发生变化时，模型的预测结果也会随之改变，进而导

致损失函数的大小发生变化。因此，更准确地说，损失函数实际上应该表示为：

$$\mathcal{L}(\theta)$$

其中，θ 表示模型中的所有参数。我们的目标，就是寻找一组最优参数 θ^*，使得损失函数达到最小：

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

这里的 θ^* 表示使损失函数取得最小值的参数组合，也就是模型希望学习到的参数。

在数学中，寻找函数最小值的一种常见方法就是**求导**。但是，利用导数求函数最小值，要求函数具有良好的可导性。绝对值函数虽然能够表示预测误差的大小，但它在零点处不可导。这意味着在损失函数为零的附近，无法通过导数准确描述函数的变化情况，不利于后续基于梯度的方法进行参数更新。

因此，我们需要一个既能够衡量预测误差，又具有良好可导性的函数。一个直接的做法，就是将误差进行平方：

$$\mathcal{L} = (y_{\text{pred}} - y_{\text{true}})^2$$

平方操作不仅消除了误差的正负方向，使损失值始终为正，同时也使函数变得连续且处处可导，更适合通过梯度下降的方法寻找最优参数。除此之外，平方还会进一步放大较大的误差。例如，当预测误差为 1 时，平方后的损失为 1；而当误差增加到 10 时，损失会变为 100。这意味着模型在训练过程中会更加关注那些偏差较大的预测结果，从而优先修正严重错误。

对于包含多个样本的数据集，我们通常不会只关注单个样本的损失，而是需要综合考虑所有样本的预测误差。因此，可以将每个样本的平方误差进行累加，并计算其平均值，这就是**均方误差（Mean Squared Error, MSE）**：

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right)^2$$

其中，n 表示样本数量。这个值反映了模型在整个数据集上的整体预测误差，数值越小，说明模型的预测结果整体越接近真实目标。

需要注意的是，在实际的训练过程中，我们通常会在平方误差前加入一个 $\frac{1}{2}$，即：

$$\mathcal{L} = \frac{1}{2} (y_{\text{pred}} - y_{\text{true}})^2$$

这里的 $\frac{1}{2}$ 主要是为了后续求导更加方便。因为对平方项求导时会产生一个 2，正好可以和前面的 $\frac{1}{2}$ 抵消：

$$\frac{\partial \mathcal{L}}{\partial y_{\text{pred}}} = \frac{\partial}{\partial y_{\text{pred}}} \left[\frac{1}{2} (y_{\text{pred}} - y_{\text{true}})^2 \right] = y_{\text{pred}} - y_{\text{true}}$$

因此，在后续的推导中，我们通常会保留这个 $\frac{1}{2}$。对于整个数据集，如果同样加入 $\frac{1}{2}$，则可以写成：

$$\mathcal{L} = \frac{1}{2n} \sum_{i=1}^{n} \left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right)^2$$

损失函数与代价函数

在机器学习中，**损失函数（Loss Function）**和**代价函数（Cost Function）**通常用于衡量不同范围内的预测误差。一般来说，损失函数用于衡量单个样本的预测误差。例如，对于第 i 个样本，其平方损失可以写成：

$$\mathcal{L}^{(i)} = \frac{1}{2}\left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)}\right)^2$$

而代价函数用于衡量整个数据集上的总体误差，通常是所有样本损失的平均值：

$$\mathcal{J} = \frac{1}{n}\sum_{i=1}^{n}\mathcal{L}^{(i)}$$

因此，可以简单理解为：损失函数用来衡量单个样本的预测误差，而代价函数用来衡量整个数据集的总体误差。为了便于后续学习，本书不做区分，统一使用损失函数这一称呼。

2.2 激活函数

我们现在知道，可以通过求导来寻找损失函数的最小值。因此，模型的计算过程需要具有良好的可导性。我们回顾一下之前设计的多层感知机模型，如图 2.1所示。

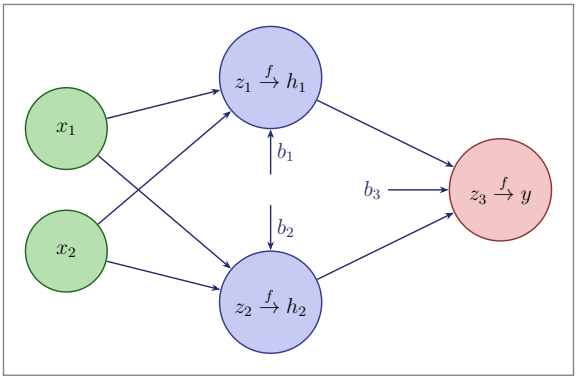


图 2.1 采用阶跃函数作为激活函数的多层感知机模型。节点内的 f 代表阶跃函数，它将神经元的线性输入 z 转换为阶跃输出（h 或 y）。

当时采用的激活函数是阶跃函数，其函数图像及对应的导数图像如图 2.2所示：

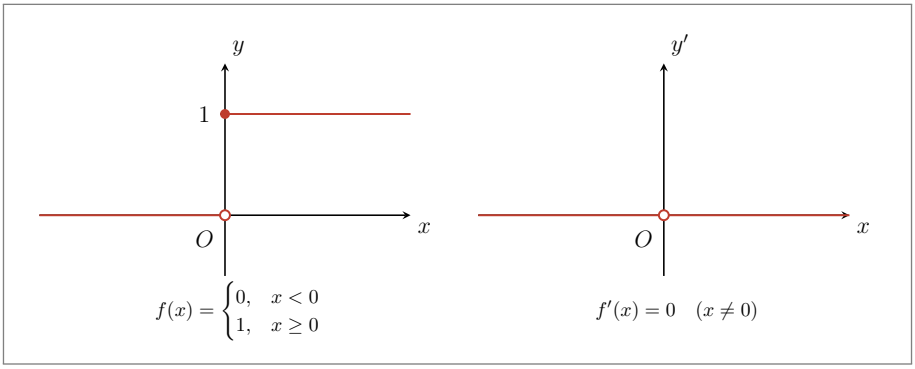


图 2.2 阶跃函数及其导数示意图。左图为阶跃函数 $f(x)$，在 $x = 0$ 处发生单位跳变；右图为其导数 $f'(x)$，在 $x \neq 0$ 的区域恒为零，在 $x = 0$ 处无定义。

阶跃函数虽然能够根据输入判断神经元是否被激活，但它存在一个明显的问题：无法提供有效的梯度信息。阶跃函数的导数在 $x \neq 0$ 的区域内恒为 0。这意味着在利用梯度下降更新参数时，相关梯度会因为乘以 0 而变成 0，参数无法得到有效的更新。而在 $x = 0$ 的位置，阶跃函数不可导，无法计算确定的梯度。

那么，能不能设计一种函数，在保留阶跃函数基本形状的同时，让输出随输入平滑变化，并且处处可导呢？实际上，数学中早就有一种函数，能够满足这些要求，这就是 **Logistic 函数（Logistic Function）**。它的输出会随着输入从较低水平逐渐过渡到较高水平，但整个变化过程是连续且平滑的，并且其导数形式简洁，易于计算。

在深度学习领域，我们通常将这个函数称为 **Sigmoid 函数（Sigmoid Function）**。“Sigmoid” 一词来源于希腊字母 σ（Sigma）的末写形式 ς，其形状与英文字母 “S” 相似。因此，在数学中，Sigmoid 函数通常泛指曲线呈 S 形的函数，而在深度学习中，Sigmoid 函数一般默认指 Logistic 函数[1]，其数学形式为：

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

如图 2.3所示，Sigmoid 函数的输出范围被限制在 0 到 1 之间。当输入值较小时，函数输出逐渐接近 0；当输入值较大时，函数输出逐渐接近 1；而在输入接近 0 的位置，函数输出会发生较为明显的变化。因此，从形状上看，Sigmoid 函数可以理解为阶跃函数的一种平滑近似。与阶跃函数在临界点处发生突然跳变不同，Sigmoid 函数通过连续、平滑的变化过程，将任意实数输入映射到 0 和 1 之间。

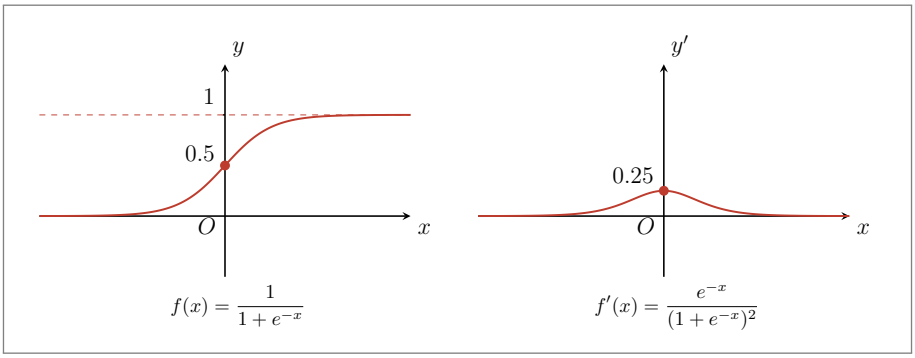


图 2.3 Sigmoid 函数及其导数示意图。左图为 Sigmoid 函数 $f(x)$，呈现平滑的 S 形曲线，输出范围限制在 0 到 1 之间；右图为其导数 $f'(x)$，在整个定义域内连续且可导。

这种映射方式也使 Sigmoid 函数具有较为直观的解释。在二分类任务中，如果将 Sigmoid 函数的输出作为模型的预测值，那么输出越接近 1，通常表示模型越倾向于判断样本属于正类；输出越接近 0，则表示模型越倾向于判断样本属于负类。

更重要的是，Sigmoid 函数在整个定义域内连续且可导，而且其导数形式也十分简洁。

$$\sigma(x) = \frac{1}{1 + e^{-x}} = (1 + e^{-x})^{-1},$$

$$\sigma'(x) = -1 \cdot (1 + e^{-x})^{-2} \cdot (-e^{-x})$$

[1] Logistic 函数作为神经网络激活函数的使用，通常可追溯至 David Rumelhart、Geoffrey Hinton 和 Ronald Williams 于 1986 年发表的论文*Learning representations by back-propagating errors*。

$$
\begin{aligned}
&= \frac{e^{-x}}{(1+e^{-x})^2} \\
&= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \\
&= \sigma(x)\,(1-\sigma(x))\,.
\end{aligned}
$$

因此，Sigmoid 函数的导数为

$$\sigma'(x) = \sigma(x)\big(1-\sigma(x)\big)$$

可以看到，Sigmoid 函数的导数可以直接利用函数本身的输出来表示，而不需要重新计算复杂的指数表达式。这种简洁的导数形式使梯度计算更加方便。

因此，从阶跃函数到 Sigmoid 函数的变化，不仅使神经元的输出从离散跳变变为连续变化，更重要的是提供了可导的数学形式，使梯度能够通过反向传播逐层传递，从而根据梯度不断调整网络参数，实现模型的学习。

2.3 反向传播算法

我们现在已经有了衡量预测误差的损失函数 $\mathcal{L}$，也用平滑可导的 Sigmoid 函数替换了不可导的阶跃函数，扫清了数学上的障碍。现在，万事俱备，只欠东风——如果模型预测得不准确，我们该如何调整网络中的参数，使损失函数逐渐减小?

我们以图 2.4所示的感知机模型为例。该模型包含两个输入节点 x_1, x_2、两个隐藏节点 h_1, h_2 以及一个输出节点 y_{pred}。不过不同于之前的设计，现在所有的激活函数均采用 Sigmoid 函数。

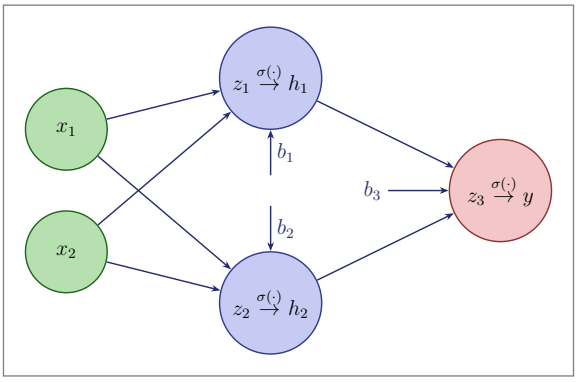


图 2.4 采用 Sigmoid 函数作为激活函数的多层感知机模型。节点内的 $\sigma(\cdot)$ 代表 Sigmoid 函数，它将神经元的线性输入 z 平滑转换为非线性输出（h 或 y），保证了模型在整个定义域内的连续可导性。

其完整的数学表达式为：

$$
\begin{aligned}
\text{隐藏层 1：}\ & z_1 = w_{11}x_1 + w_{12}x_2 + b_1 \quad \Rightarrow \quad h_1 = \sigma(z_1) \\
\text{隐藏层 2：}\ & z_2 = w_{21}x_1 + w_{22}x_2 + b_2 \quad \Rightarrow \quad h_2 = \sigma(z_2) \\
\text{输出层：}\ & z_3 = v_1 h_1 + v_2 h_2 + b_3 \quad \Rightarrow \quad y_{\text{pred}} = \sigma(z_3)
\end{aligned}
$$

激活函数：$\sigma(z) = \dfrac{1}{1+e^{-z}}$

损失函数：$\mathcal{L} = \dfrac{1}{2}(y_{\text{pred}} - y_{\text{true}})^2$

首先，我们进行**参数初始化（Parameter Initialization）**，即为网络中的权重和偏置设置初始值。我们仍然采用之前异或问题的参数设置，将所有权重初始化为 1，所有偏置初始化为 0。

随后，我们进行**前向传播（Forward Propagation）**，即将输入数据送入模型中，通过隐藏层和输出层逐层计算，最终得到模型的预测结果。由于异或问题属于二分类任务，我们以阈值 0.5 作为分类边界：当模型输出大于 0.5 时，预测为 1；否则预测为 0。经过前向传播后，四组输入对应的预测结果如表 2.1所示：

表 2.1 采用 Sigmoid 激活函数的异或问题预测结果

输入 x_1	输入 x_2	模型输出 y	期望输出	是否正确（阈值 0.5）
0	0	0.7311	0	**错误**
1	1	0.8534	0	**错误**
0	1	0.8119	1	正确
1	0	0.8119	1	正确

可以看到，结果非常糟糕。在这组初始参数下，即使采用了平滑可导的 Sigmoid 函数，模型仍然无法正确解决异或问题。四组输入的输出都大于 0.5，因此模型把它们全部预测成了 1。

接下来，我们需要进一步**计算模型的损失**。对于任意第 i 个样本（i 从 1 到 4），模型前向传播得到的预测输出为 $y_{\text{pred}}^{(i)}$，真实标签为 $y_{\text{true}}^{(i)}$，对应的单点损失为

$$\mathcal{L}^{(i)} = \frac{1}{2}\left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)}\right)^2$$

为了评估模型在整个数据集上的表现，我们不能只关注某一个样本，而需要综合考虑全部四个样本。因此，将四个样本的损失取平均，得到整体的损失（准确地说，这里应该称为代价函数）：

$$\mathcal{J} = \frac{1}{4}\sum_{i=1}^{4}\mathcal{L}^{(i)} = \frac{1}{4}\sum_{i=1}^{4}\frac{1}{2}\left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)}\right)^2$$

现在，我们的目标是不断调整模型中的参数，使整体损失 $\mathcal{J}$ 不断减小，并最终找到一组参数，使其最小。为此，我们需要对损失函数进行**求导**，更准确地说，是计算损失函数关于各个模型参数的**梯度（Gradient）**。这里所说的模型参数，也就是整体损失 $\mathcal{J}$ 的自变量，指的是模型中需要通过训练不断调整的权重和偏置，而不是输入数据 x_1, x_2。对于当前的训练样本，输入数据 x_1, x_2 以及对应的真实标签都是已经给定的，我们真正需要优化的是网络中的各个权重和偏置。

由于整体损失 $\mathcal{J}$ 是各个样本损失的平均，我们可以先从单个样本入手，计算其损失 $\mathcal{L}$ 关于网络参数的偏导数，再将各个样本得到的梯度求平均。因此，下面我们暂时只考虑一个样

本，并以隐藏层参数 w_{11} 为例，计算损失 $\mathcal{L}$ 关于该参数的偏导数：

$$\frac{\partial \mathcal{L}}{\partial w_{11}}$$

梯度

所谓梯度，就是将损失函数关于每一个参数的偏导数组合起来，用来描述损失随各个参数变化的方向和程度。

$$\frac{\partial \mathcal{L}}{\partial w_{11}}$$

只是关于单个参数 w_{11} 的一个偏导数。由于网络中还存在其他参数，例如 w_{12}，w_{21}，w_{22}，v_1，v_2，b_1，b_2，b_3，因此还需要分别计算损失关于这些参数的偏导数，并将它们组合起来，形成完整的梯度。

例如，对于参数集合

$$\boldsymbol{\theta} = (w_{11}, w_{12}, w_{21}, w_{22}, v_1, v_2, b_1, b_2, b_3)$$

损失函数 $\mathcal{L}$ 关于这些参数的梯度可以写成：

$$\nabla_{\boldsymbol{\theta}} \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial w_{11}}, \frac{\partial \mathcal{L}}{\partial w_{12}}, \frac{\partial \mathcal{L}}{\partial w_{21}}, \frac{\partial \mathcal{L}}{\partial w_{22}}, \frac{\partial \mathcal{L}}{\partial v_1}, \frac{\partial \mathcal{L}}{\partial v_2}, \frac{\partial \mathcal{L}}{\partial b_1}, \frac{\partial \mathcal{L}}{\partial b_2}, \frac{\partial \mathcal{L}}{\partial b_3} \right)$$

然而，损失 $\mathcal{L}$ 并不是 w_{11} 的直接函数，无法直接求出 $\frac{\partial \mathcal{L}}{\partial w_{11}}$。回顾模型的前向传播过程，$w_{11}$ 首先参与计算隐藏层的总输入：

$$z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$

因此，根据链式法则，可以先将损失对 w_{11} 的偏导拆开：

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = \frac{\partial \mathcal{L}}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}}$$

由于 z_1 关于 w_{11} 的偏导数为 x_1，所以：

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = x_1 \cdot \frac{\partial \mathcal{L}}{\partial z_1}$$

我们接着求 $\frac{\partial \mathcal{L}}{\partial z_1}$，但 $\mathcal{L}$ 同样不是 z_1 的直接函数。由于 z_1 经过 Sigmoid 函数得到隐藏层输出 h_1，即 $h_1 = \sigma(z_1)$，我们可以再次使用链式法则：

$$\frac{\partial \mathcal{L}}{\partial z_1} = \frac{\partial \mathcal{L}}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1}$$

其中，$\frac{\partial h_1}{\partial z_1}$ 就是 Sigmoid 函数在 z_1 处的导数，所以：

$$\frac{\partial h_1}{\partial z_1} = h_1 \cdot (1 - h_1)$$

于是，问题进一步转化为求 $\frac{\partial \mathcal{L}}{\partial h_1}$。由于 h_1 参与计算输出层总输入 z_3，即 $z_3 = v_1 h_1 + v_2 h_2 + b_3$，因此：

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1}$$

换句话说，参数 w_{11} 影响了 z_1，z_1 决定了隐藏神经元 h_1 的输出，h_1 又通过输出层参与计算 z_3，进而影响最终输出 y_{pred}，而 y_{pred} 决定了最终损失值 $\mathcal{L}$。因此，w_{11} 对最终损失的

影响路径为：

$$w_{11} \to z_1 \to h_1 \to z_3 \to y_{\text{pred}} \to \mathcal{L}$$

沿着这条路径，将各部分的导数依次连接起来，就可以得到完整的链式法则：

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = \frac{\partial \mathcal{L}}{\partial y_{\text{pred}}} \cdot \frac{\partial y_{\text{pred}}}{\partial z_3} \cdot \frac{\partial z_3}{\partial h_1} \cdot \frac{\partial h_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}}$$

这正是**反向传播 (Backpropagation)** 的核心思想：利用链式法则，从损失函数出发，沿着网络计算路径的反方向逐层传播梯度，最终得到损失函数关于各个模型参数的偏导数，从而为参数更新提供依据。观察刚才得到的导数链可以发现，要计算网络较前面的参数 w_{11} 对损失的影响，需要依次经过 y_{pred}、z_3、h_1 和 z_1 等中间变量。也就是说，越靠近网络前端的参数，其梯度计算越依赖后面节点的计算结果。因此，与其从参数 w_{11} 出发一路向后追踪，不如从已经确定的损失 $\mathcal{L}$ 开始，沿着计算路径反方向逐层计算梯度，并将已经得到的中间结果逐层传递下去。

正如图 2.5所示，前向传播负责从输入出发，依次经过网络各层计算预测结果和损失；反向传播则从损失出发，沿着与前向传播相反的方向逐层传递梯度。正因为这种计算过程是沿网络计算路径反向进行的，所以称为反向传播（Backpropagation）。

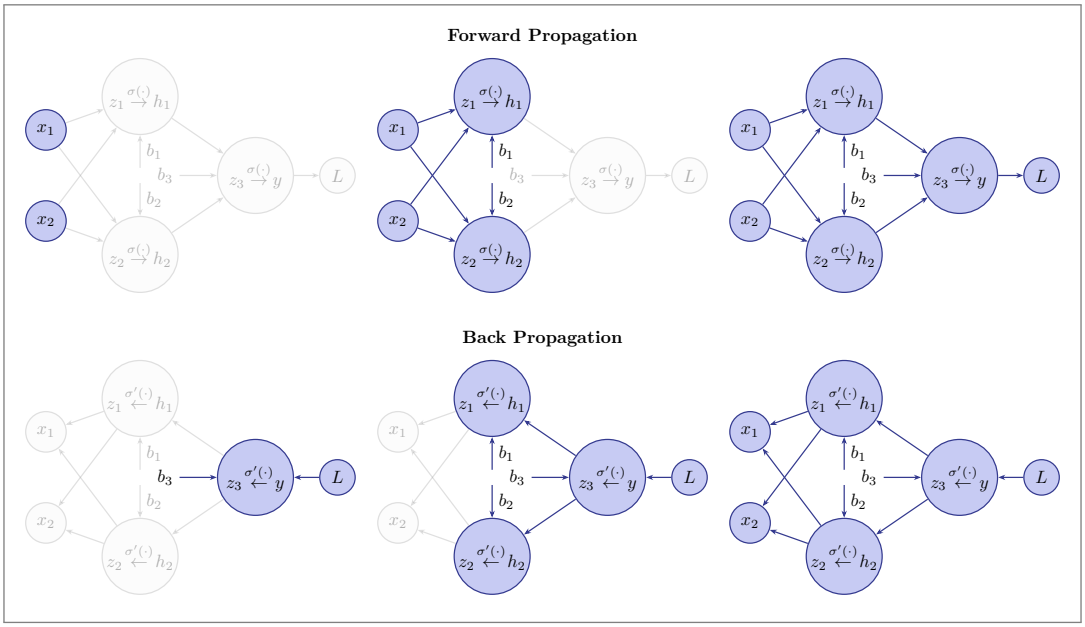


图 2.5 多层感知机的前向传播与反向传播示意图。上排为前向前向传播：输入经隐藏层、输出层逐层算出损失 L；下排为反向传播：梯度自 L 沿计算路径反方向回传至输入。图中高亮表示本步已激活的节点与连接，半透明部分表示尚未涉及。

明确了反向传播的基本思想后，我们来看一个具体的计算过程。由于整体损失 $\mathcal{J}$ 是各个样本损失的平均，我们先选取异或问题中的一个样本 $(1, 1)$，计算其单个损失 $\mathcal{L}$ 关于参数 w_{11} 的偏导数。之前得到的计算路径为：

$$w_{11} \to z_1 \to h_1 \to z_3 \to y_{\text{pred}} \to \mathcal{L},$$

首先，从输出层开始。损失关于输出层总输入 z_3 的偏导数为：

$$\delta_3 = \frac{\partial \mathcal{L}}{\partial z_3} = (y_{\text{pred}} - y_{\text{true}})\, y_{\text{pred}}(1 - y_{\text{pred}})$$

对于样本 $(1,1)$，前向传播得到 $y_{\text{pred}} \approx 0.8534$、$y_{\text{true}} = 0$，因此 $\delta_3 \approx 0.1068$。接下来，将 δ_3 沿连接权重 v_1 反向传递到隐藏层，并结合隐藏层 Sigmoid 函数的导数，可以得到隐藏层的局部误差信号：

$$\delta_1 = \frac{\partial \mathcal{L}}{\partial z_1} = \delta_3\, v_1\, h_1(1 - h_1) \approx 0.0112$$

最后，由于 z_1 关于 w_{11} 的偏导数为 x_1，所以：

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = \delta_1 x_1 = (y_{\text{pred}} - y_{\text{true}}) y_{\text{pred}}(1 - y_{\text{pred}}) \cdot v_1 \cdot h_1(1 - h_1) \cdot x_1$$

对于样本 $(1,1)$，$x_1 = 1$，所以：

$$\frac{\partial \mathcal{L}}{\partial w_{11}} \approx 0.0112 \times 1 = 0.0112.$$

这样，我们就完成了一个样本的反向传播计算：从损失 $\mathcal{L}$ 出发，先得到输出层误差 δ_3，再将其传递到隐藏层得到 δ_1，最后计算出参数 w_{11} 的梯度。

2.4 梯度下降算法

现在，我们已经能够利用反向传播算法计算出损失函数关于各个参数的梯度了。但是，仅仅算出梯度是不够的，我们还需要利用它来更新模型参数，使损失逐渐减小。那么，梯度究竟应该如何指导参数的更新呢？为了直观理解模型如何利用梯度进行学习，我们先将问题简化为只有一个参数 θ 的情况，如图 2.6所示。

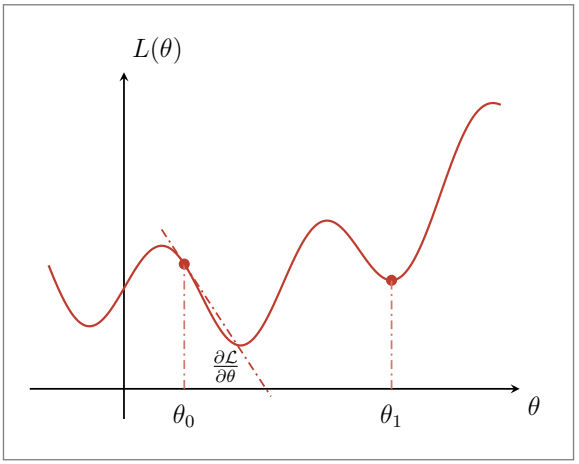


图 2.6 损失函数曲线与梯度下降示意图。横轴为参数 θ，纵轴为损失 $\mathcal{L}(\theta)$，θ_0 处虚线切线的斜率即该点梯度 $\partial\mathcal{L}/\partial\theta$。

我们当前的目标是寻找一个合适的 θ，使得 $\mathcal{L}(\theta)$ 尽可能小，也就是寻找损失函数曲线的最低点。在图 2.6中，曲线在某一点处的斜率，就是损失函数关于参数 θ 在该点的导数。当 θ 位于最低点左侧时，导数为负，曲线向右下降；当 θ 位于最低点右侧时，导数为正，曲线向右上升。

因此，我们可以根据导数的正负来判断参数应该向哪个方向调整。当

$$\frac{\partial \mathcal{L}}{\partial \theta} > 0$$

时，说明 θ 增大会使得损失增大，因此应该减小 θ；反之，当

$$\frac{\partial \mathcal{L}}{\partial \theta} < 0$$

时，说明 θ 增大会使得损失减小，因此应该增大 θ。也就是说，要让损失函数朝减小的方向变化，我们应该沿着梯度的反方向调整参数。

为了控制每次参数调整的幅度，我们引入**学习率（Learning Rate）**这一超参数，通常记作 η，它决定了每次参数更新迈出的步伐大小。将梯度下降的思想写成数学形式，就得到**梯度下降算法（Gradient Descent）**的参数更新公式：

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

其中，$\frac{\partial \mathcal{L}}{\partial \theta}$ 表示当前参数处的梯度，负号表示沿着梯度的反方向更新，而 η 则控制更新的幅度。只要学习率选择合适，经过不断迭代，参数就能够逐渐向损失较小的区域移动。

我们回到之前的多层感知机模型，看看梯度下降如何更新模型参数。在前面的反向传播示例中，我们以输入样本 $(1, 1)$ 为例，计算得到了该样本损失 $\mathcal{L}$ 关于权重 w_{11} 的梯度：

$$\frac{\partial \mathcal{L}}{\partial w_{11}} = 0.0112.$$

我们现在使用这个样本计算得到的梯度来更新参数，并设当前权重 $w_{11} = 1$，学习率 $\eta = 1$。根据梯度下降公式：

$$\begin{aligned} w_{11}^{\text{new}} &= w_{11}^{\text{old}} - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_{11}} \\ &= 1 - 1 \times 0.0112 \\ &= 0.9888 \end{aligned}$$

由于当前梯度为正，梯度下降将 w_{11} 向减小的方向调整，使其从 1 变为 0.9888。这就是梯度真正发挥作用的方式：反向传播负责计算梯度，而梯度下降则根据梯度更新模型参数。

至此，我们就把神经网络的一次完整学习过程串联起来了。首先，模型通过前向传播得到预测结果并计算损失，接着通过反向传播计算各个参数的梯度，最后利用梯度下降更新模型参数。具体而言，一次参数更新可以概括为：

$$\text{前向传播} \rightarrow \text{计算损失} \rightarrow \text{反向传播} \rightarrow \text{计算梯度} \rightarrow \text{梯度下降} \rightarrow \text{更新参数}$$

通过不断重复这一过程，模型参数逐渐得到调整，使损失整体趋于减小。

反向传播算法发展史

反向传播算法是现代神经网络训练的核心基石，但它的发展历程并非一蹴而就，而是经历了数十年的沉淀与曲折。

早在 1967 年，日本学者 Shunichi Amari 便对自适应模式分类器中的学习机制进行了系统研究，并在论文*A Theory of Adaptive Pattern Classifiers*中详细阐述了利用梯度调

整模型参数的思想。这一工作在数学本质上与后来的反向传播算法高度契合。然而，受限于当时匮乏的计算能力以及神经网络研究的边缘化处境，这些早期成果并没有得到广泛关注。

1974 年，哈佛大学研究生 Paul Werbos 在其博士毕业论文*Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*中，首次系统地提出了利用误差反向传播训练多层网络的方法。遗憾的是，彼时正值第一次人工智能寒冬，神经网络研究陷入低谷，这项极具前瞻性的工作同样被学术界束之高阁。

直到 1986 年，David Rumelhart、Geoffrey Hinton 和 Ronald Williams 在《Nature》上发表了具有里程碑意义的论文*Learning representations by back-propagating errors*。论文系统阐述了误差反向传播算法在多层神经网络中的训练机制，并通过实验证明，网络能够通过梯度下降不断调整权重，使隐藏层逐步提取出任务相关的特征表示。这项工作有效突破了多层网络难以训练的瓶颈，极大地推动了反向传播算法的普及，使其正式确立了作为神经网络核心训练方法的地位。

2.5 计算图与自动微分

我们以图 2.7所示的多层感知机模型为例，手工推导了反向传播的过程。可以发现，损失函数关于某个参数的梯度，并不是直接计算出来的，而是沿着模型中的计算路径，依据链式法则将各个局部导数依次相乘得到的。

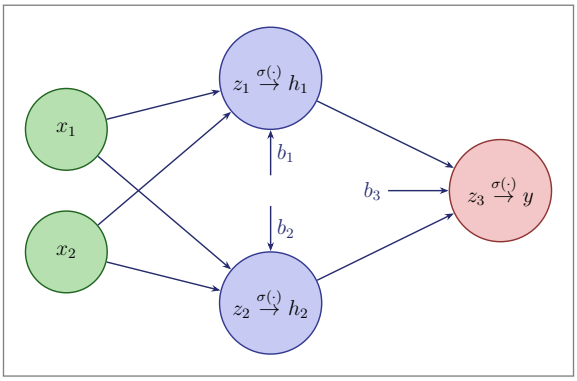


图 2.7 采用 Sigmoid 函数作为激活函数的多层感知机模型。节点内的 $\sigma(\cdot)$ 代表 Sigmoid 函数，它将神经元的线性输入 z 平滑转换为非线性输出（h 或 y），保证了模型在整个定义域内的连续可导性。

之前的例子中，参数 w_{11} 影响最终损失的过程可以表示为：

$$w_{11} \to z_1 \to h_1 \to z_3 \to y_{\text{pred}} \to \mathcal{L}.$$

这条路径实际上就是模型前向计算过程的一部分：w_{11} 参与计算 z_1，z_1 经过 Sigmoid 函数得到 h_1，随后 h_1 参与输出层的计算，最终得到预测结果 y_{pred} 和损失 $\mathcal{L}$。如果将网络中的所有变量及其计算关系按照这种依赖关系连接起来，就得到了整个模型的**计算图（Computational Graph）**，如图 2.8所示。

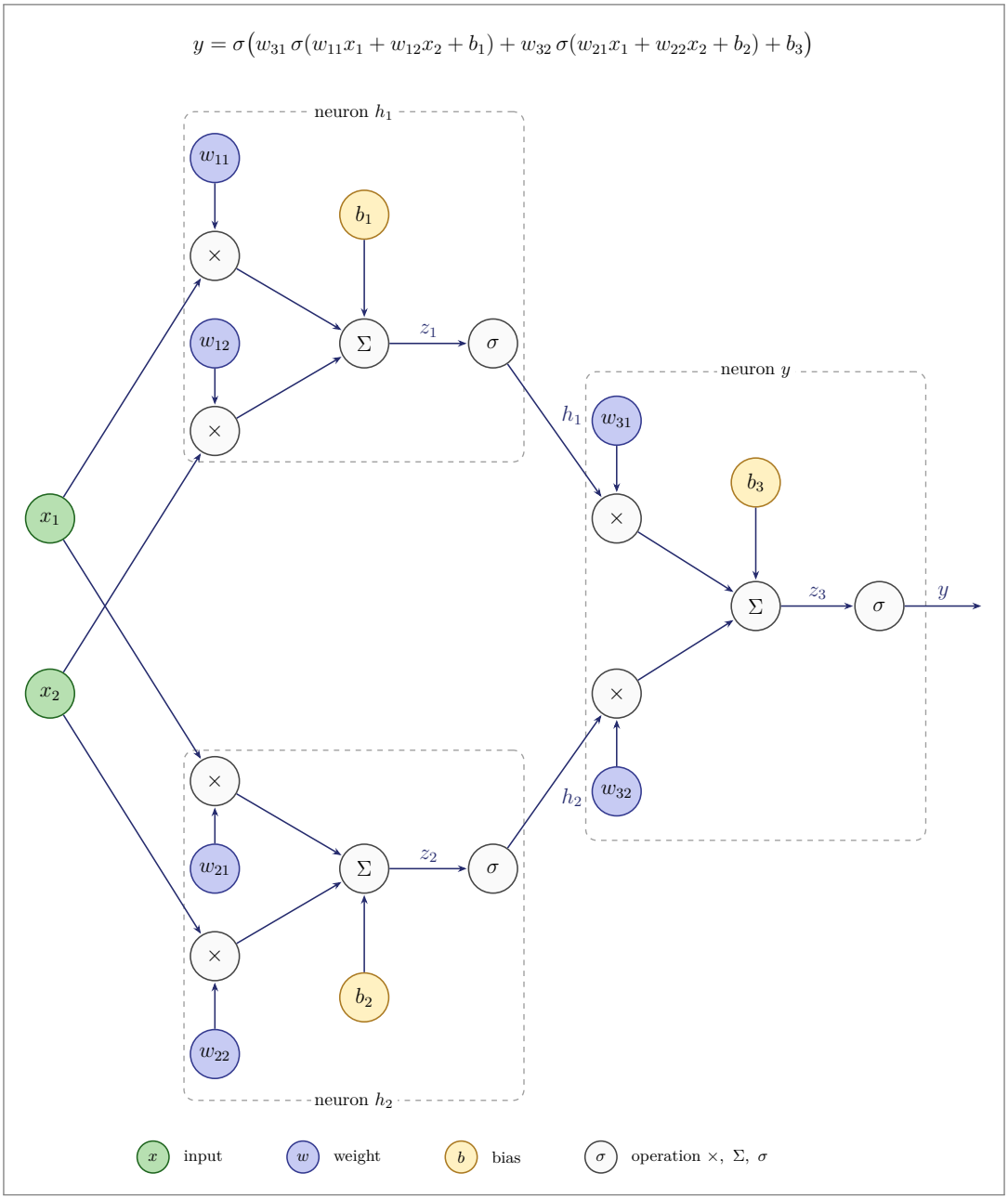


图 2.8 多层感知机模型的计算图。图中节点表示输入、参数、偏置与基本运算（×、Σ、σ），虚线框标出各个神经元。

有了计算图之后，模型的前向传播和梯度计算都可以在这张图上完成。前向传播时，从输入出发，按照计算关系依次计算各个节点的数值；计算梯度时，则从最终的损失出发，沿着相反的方向逐层回传梯度，并依据链式法则完成各个局部导数的计算。在实际项目中，我们不需要再针对每个模型、每个参数手动推导梯度。以 `PyTorch` 为例，只需要在前向计算得到损失后调用 `loss.backward()`，框架就会根据计算图自动完成梯度计算，并将相应的梯度保存到各个参数中，这就是**自动微分（Automatic Differentiation）**。也就是说，**计算图负责记**

录计算过程，自动微分负责根据计算图自动计算梯度。

深度学习框架

在实际项目中，我们通常不会手动完成神经网络中的梯度计算、参数更新以及大量底层计算，而是会借助**深度学习框架 (Deep Learning Framework)** 来完成这些工作。当前常见的深度学习框架包括 `PyTorch`、`TensorFlow` 等，它们为模型构建、自动微分、参数优化以及 GPU 计算等提供了完整的工具支持。下面介绍深度学习框架的三项核心功能。

第一，自动微分。前面我们已经介绍过，自动微分可以根据前向传播过程中记录的计算关系，自动计算损失函数关于各个模型参数的梯度。在 `PyTorch` 中，我们只需要在得到损失后调用 `loss.backward()`，就可以自动完成梯度计算，而不需要再手动推导每一个参数的梯度。

第二，张量计算与 GPU 加速。我们一直以单个神经元的形式描述模型，但在实际计算中，多个神经元的计算可以统一组织成矩阵运算，从而一次性完成大量计算。以之前的多层感知机为例：

$$y = \sigma\big(w_{31}\sigma(w_{11}x_1 + w_{12}x_2 + b_1) + w_{32}\sigma(w_{21}x_1 + w_{22}x_2 + b_2) + b_3\big)$$

我们可以将权重、偏置和中间变量统一表示为矩阵和向量，并将整个前向传播过程写成如下矩阵形式：

$$y = \mathbf{v}^\top \sigma\left(\begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}\right) + b_3$$

其中，各矩阵与向量分别为：

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad \mathbf{W} = \begin{pmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} w_{31} \\ w_{32} \end{pmatrix}$$

这样一来，原本需要逐个神经元计算的过程，现在可以统一转换为矩阵运算。当模型需要同时处理多个样本时，还可以将多个输入组成一个矩阵，通过批量矩阵运算一次性完成整批数据的计算。而矩阵运算具有高度的并行性，非常适合 GPU 执行，因此现代深度学习框架通常利用 GPU 加速大规模矩阵运算。在 `PyTorch` 中，只需要使用 `.to('cuda')`，即可将模型和数据移动到 GPU。

第三，模块化建模与训练。深度学习框架将模型构建和训练过程中常用的组件封装成可以直接调用的模块，开发者不需要从零实现每一个基础功能。在 `PyTorch` 中，常见的损失函数都已经提供了现成实现，我们只需要根据任务选择合适的损失函数即可。例如，均方误差损失可以直接通过 `nn.MSELoss()` 调用，平均绝对误差则可以通过 `nn.L1Loss()` 调用。同时，模型的训练模式也可以通过调用 `.train()` 直接切换，无需手动处理相关设置，从而更加方便地完成模型的构建与训练。

2.6 万能逼近定理

经过前面的学习，我们已经知道神经网络能够通过训练不断调整参数，从而学习数据中的规律。那么，神经网络的表达能力到底有多强？除了能够解决异或这样的非线性问题，它还能表示多么复杂的函数？

答案是：在一定条件下，神经网络几乎可以逼近任意复杂的连续函数，这就是**万能逼近定理 (Universal Approximation Theorem)**。所谓万能逼近定理 (Universal Approximation Theorem)，可以通俗地理解为：对于定义在紧集上的任意连续函数 f，以及任意给定的精度 $\epsilon > 0$，只要单隐藏层前馈神经网络包含足够多的神经元，就可以使网络输出与 f 之间的误差小于 ϵ。也就是说，一个足够宽的神经网络，原则上可以逼近任意复杂的连续函数。这一结论最早由多位研究者在 1989 年前后独立证明。1989 年，George Cybenko 在论文*Approximation by Superpositions of a Sigmoidal Function*中证明了，采用连续 Sigmoid 激活函数的单隐藏层前馈神经网络可以以任意精度逼近定义在单位超立方体上的连续函数。同年，日本学者 Ken-Ichi Funahashi 也独立给出了类似的结果，在论文*On the Approximate Realization of Continuous Mappings by Neural Networks*中证明了采用 Sigmoid 激活函数的多层神经网络能够近似实现任意连续映射。此后，Kurt Hornik、Maxwell Stinchcombe 和 Halbert White 在论文*Multilayer Feedforward Networks are Universal Approximators*中进一步推广了这一结论，将万能逼近性质扩展到更广泛的激活函数类别。对于有界、非恒定且连续的激活函数，单隐藏层前馈神经网络同样能够在紧集上以任意精度逼近任意连续函数。

这里我们不展开万能逼近定理的具体数学证明，因为其中涉及的数学工具已经超出了本书的范围。需要注意的是，这个定理本质上是一个存在性定理：它只告诉我们，在满足条件的情况下，存在一个足够宽的神经网络可以逼近目标函数，却没有告诉我们应该如何找到这样一个网络。总的来说，万能逼近定理告诉我们，神经网络拥有近乎无穷的表达潜力（当然，人的潜力也是无穷的）。

我们可以通过一个直观的例子，看看神经网络是如何逼近复杂的函数。如图 2.9所示，我们希望逼近图中的连续曲线：

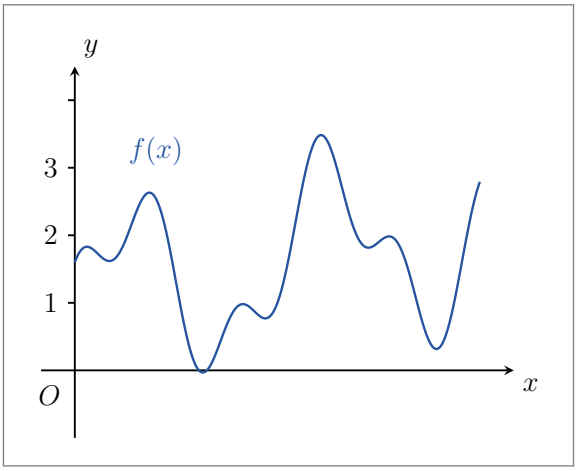


图 2.9 复杂连续函数图像。图中蓝色曲线为目标函数 $f(x)$，由整体趋势叠加高频振荡构成。

其函数式是：

$$f(x) = 1.2\sin(120x) + 0.6\cos(300x) + 0.2x + 1$$

可以看到，这个函数在整体趋势上叠加了高频振荡，具有较为复杂的变化。现在，我们使用 Sigmoid 网络来逼近它。

如图 2.10所示：当隐藏层只有 8 个神经元时，网络的表达能力有限，只能大致捕捉目标函数的整体趋势，对其中的高频变化拟合得并不理想；当神经元数量增加到 36 个后，网络可以利用更多的参数调整输出曲线，拟合效果明显提升。

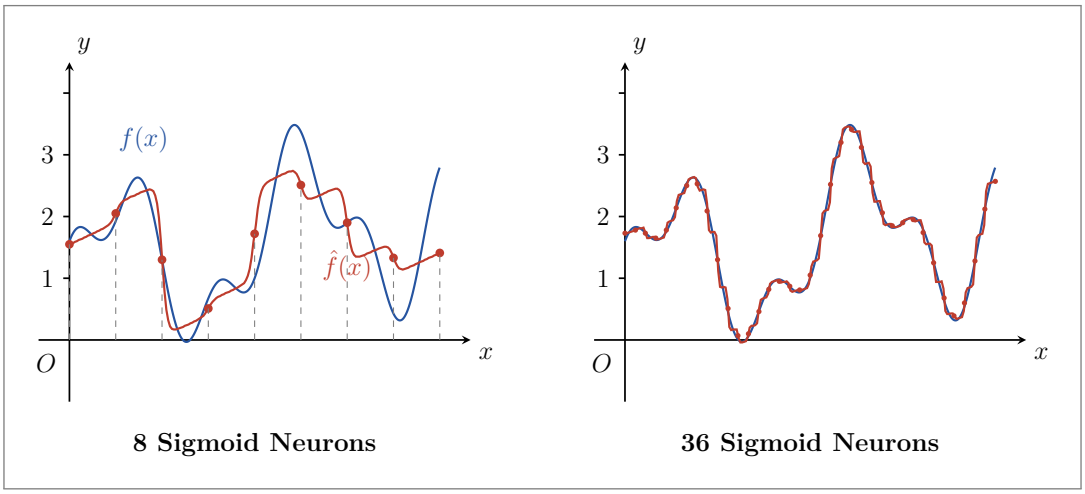


图 2.10 Sigmoid 网络对目标函数的逼近效果对比。左图隐藏层含 8 个神经元，右图含 36 个神经元；红色曲线为网络输出 $\hat{f}(x)$，蓝色曲线为目标函数 $f(x)$。

可以看到，随着神经元数量增加，网络能够表达的函数更加丰富，对目标函数的拟合也更加精细。这个例子直观展示了万能逼近定理所揭示的思想：只要网络具有足够的表达能力，就能够以任意高的精度逼近复杂的连续函数。

我们已经知道如何计算梯度，也知道如何沿着梯度下降的方向更新参数。那么，学习率应该设置多大？一次应该使用多少个样本？有没有更加高效的参数更新方法，让模型能够学得更快、更稳？

第 3 章　神经网络优化算法

我们现在已经学会了用反向传播来计算梯度，并利用梯度下降来更新模型参数。一切看起来都很完美，数学推导无懈可击，似乎我们只需要把数据喂给模型，模型就能轻而易举地找到最优解。然而，真实的深度学习训练过程并没有这么简单。

在理想情况下，我们希望损失函数的地形如图 3.1所示，呈现一个平滑的“U 型碗”状。这样的函数属于**凸函数（Convex Function）**。对于凸函数，任意局部最优解都是全局最优解，因此梯度下降能够沿着损失下降的方向不断逼近最低点，并较为稳定地收敛到全局最优解。

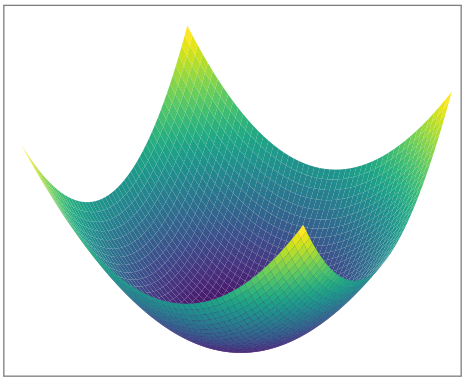


图 3.1　凸函数的损失曲面。曲面呈平滑的“U 型碗”形态，任何局部极小值均为全局最小值。

然而，真实的深度学习模型通常包含数万乃至上亿个参数，其对应的损失函数往往是复杂的**非凸函数（Non-convex Function）**，如图 3.2所示。与理想的“U 型碗”不同，非凸损失地形往往更加复杂，可能包含局部极小值、鞍点以及高曲率区域。在这样的地形中，梯度下降不再能够轻松地找到全局最优解，参数更新可能在陡峭区域来回震荡，也可能在平坦区域前进缓慢，从而使优化过程变得十分困难。

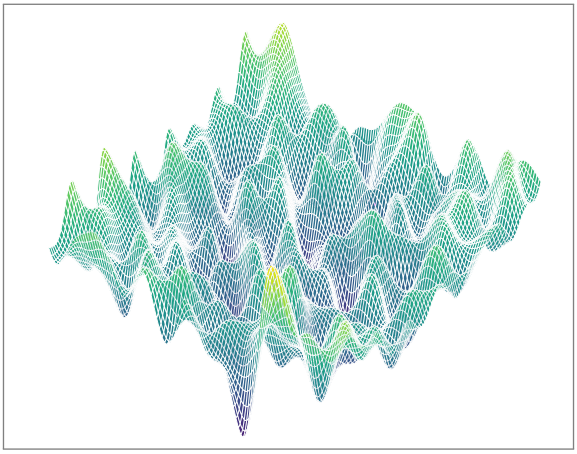


图 3.2　非凸函数的损失曲面。曲面拓扑结构复杂，充斥着局部极小值、鞍点与高曲率区域。

因此，仅靠最基础的梯度下降往往难以满足实际训练的需求。我们需要对这一参数更新公式 $\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$ 进行进一步改进，使模型能够更加高效、稳定地优化参数。

3.1 学习率

我们首先回顾一下梯度下降的核心公式：

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}$$

在这个公式中，$\frac{\partial \mathcal{L}}{\partial \theta}$ 表示损失函数对当前参数的梯度，它指向损失函数增长最快的方向；而负梯度 $-\frac{\partial \mathcal{L}}{\partial \theta}$ 就指向损失函数下降最快的方向，也是参数更新的方向。学习率 η 是步长系数，用来缩放梯度；参数每次实际更新的幅度由学习率与梯度大小共同决定。因此可以直观地理解为：负梯度决定“往哪个方向走”，学习率决定“沿这个方向走多远”。

直觉上，学习率设置得过大或者过小，都可能给模型训练带来问题，正所谓过犹不及。那么，学习率设置得过大或者过小，究竟会有什么后果呢?

图 3.3左图展示了学习率过大的情况。由于每次参数更新的幅度过大，参数很容易一次跨过 Loss 较低的位置，直接进入另一次梯度（$\frac{\partial \mathcal{L}}{\partial \theta}$）更大的区域，从而导致下一次更新的幅度进一步增大。如此反复，参数很可能在最优解两侧不断来回越过，形成剧烈震荡（Oscillation），甚至逐渐发散，使 Loss 持续增大，最终因数值溢出而出现 `NaN`（Not a Number），导致训练失败。

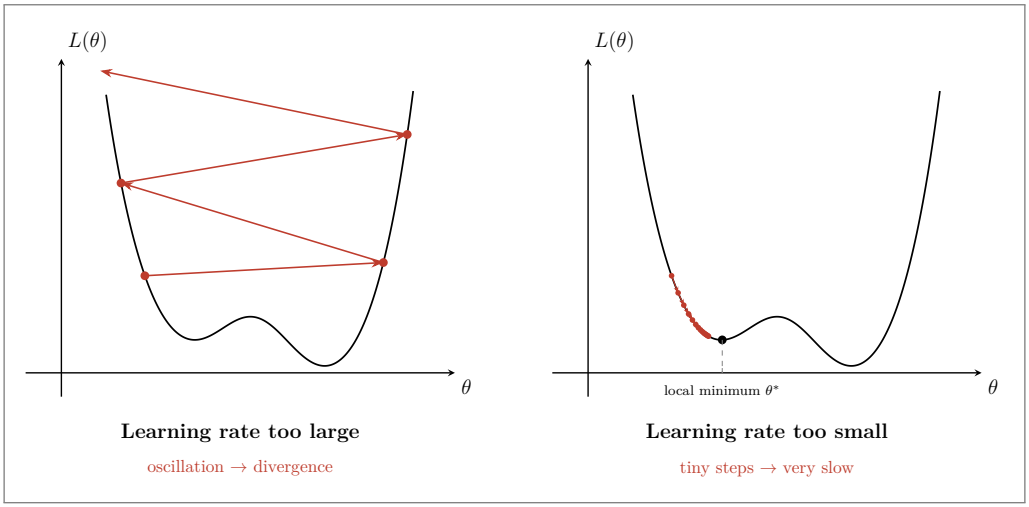


图 3.3 学习率过大与过小的对比。左图学习率过大，参数在损失曲线两侧震荡甚至发散，无法收敛；右图学习率过小，参数步长极小，向局部最小值的收敛极其缓慢。

图 3.3右图展示了学习率过小的情况。此时，每次参数更新的幅度都非常有限，参数虽然会沿着负梯度方向缓慢移动，但需要经过大量迭代才能明显降低 Loss，导致训练收敛速度过慢。更重要的是，在复杂的损失函数中，参数可能会遇到局部极小值或鞍点（Saddle Points）。而由于局部极小值和鞍点附近的梯度往往接近于 0，当学习率本身也很小时，η 与 $\frac{\partial \mathcal{L}}{\partial \theta}$ 共同作用，会使参数每次更新的幅度进一步减小。模型一旦进入这些区域，可能在整个训练周期内

都无法离开，从而使训练速度显著下降，甚至出现近似停滞的现象。

鞍点

鞍点是损失函数中的一种特殊位置：它的梯度为零，但既不是局部极小值，也不是局部极大值。

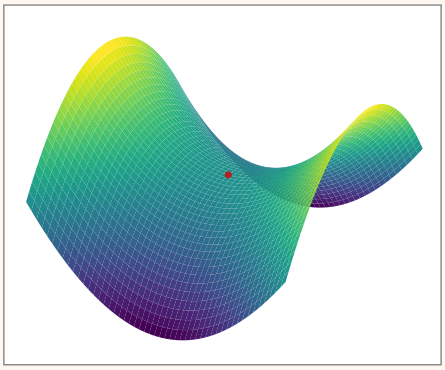


图 3.4 经典马鞍面 $z = x^2 - y^2$。曲面沿 x 轴方向向上弯曲、沿 y 轴方向向下弯曲，形似马鞍。红点为鞍点：该处梯度为零，既非局部极小值也非局部极大值。

在鞍点附近，沿着某些方向移动，Loss 会增大；而沿着另一些方向移动，Loss 反而会减小。它既不像局部极小值：在局部极小值附近，向任意方向移动，Loss 都会增大；也不像局部极大值：在局部极大值附近，向任意方向移动，Loss 都会减小。如图 3.4所示，这种曲面的形状看起来很像马鞍，所以该位置的点被称为“鞍点”。

理想的训练过程应该如图 3.5所示。在训练初期，参数距离最优解还比较远，此时可以采用较大的学习率，让参数以较大的步长快速向最优解靠近；随着训练进行，参数逐渐接近最优解，此时如果仍然保持较大的步长，就容易在最优解附近来回震荡。因此，训练后期需要逐渐减小学习率，让参数以更小的步长进行调整，从而更加稳定地逼近最优解。

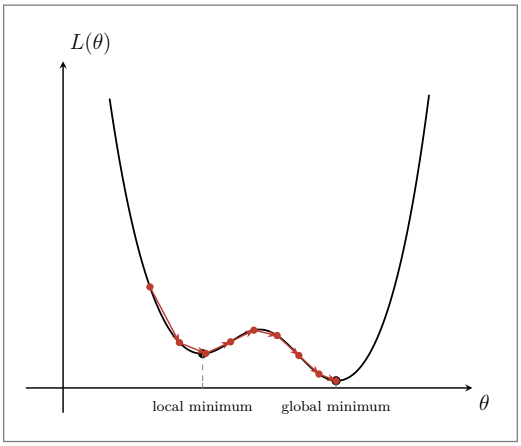


图 3.5 合理设置学习率下的梯度下降轨迹。在远离全局最小值时采用较大学习率，参数得以快速下降并越过较浅的局部极小值（local minimum）；当接近全局最小值时逐渐减小学习率，避免在谷底附近震荡，最终平稳、精确地收敛至全局最小值（global minimum）。

既然训练的不同阶段对学习率有不同的需求，那么学习率就应该随着训练过程动态调整：

训练初期保持较大，训练后期逐渐减小。

为了实现学习率随训练过程逐渐减小，一个直接的做法，就是每隔固定的训练轮次，将当前学习率乘以一个小于 1 的系数，使学习率在特定时刻快速下降。例如，每训练完 10 个 Epoch，就将学习率缩减为原来的十分之一。这种按照固定间隔逐次降低学习率的策略，就是**阶跃衰减（Step Decay）**。其数学表达式为：

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor \frac{t}{s} \rfloor}$$

其中，η_0 表示初始学习率，γ 表示衰减系数，通常取 0.1，t 表示当前的训练轮次（Epoch），s 表示衰减步长，即每经过多少个 Epoch 进行一次衰减。例如，当 $s = 10$ 时，模型每训练 10 个 Epoch，学习率就衰减一次。

可以看到，在这种策略下，学习率在一段时间内保持不变，到达设定的 Epoch 后突然下降一个台阶。训练初期，模型使用较大的学习率快速调整参数；随着训练进行，学习率逐步降低，使参数更新越来越精细。不过，这种方法也存在明显的不足：学习率的变化并不连续，而是呈阶梯状下降。每次学习率突然降低后，Loss 的下降速度也可能随之发生明显变化，使整个训练过程显得不够平滑。

为了让学习率能够更加平滑地下降，我们希望它随着训练过程连续变化，而不是在某几个时刻突然降低。同时，学习率的变化速度也应该有所区别：训练初期下降得比较缓慢，以保留较大的学习率进行快速调整；训练中期下降速度加快，使学习率逐渐减小；到了训练后期再次放缓，让模型以更小的步长进行精细调整。这种变化方式与整个训练过程比较契合。那么，有没有一种函数能够满足这样的变化规律呢？如图 3.6所示，余弦函数 $\cos(x)$ 在 $[0, \pi]$ 区间内恰好具有这样的特点：两端变化较慢，中间变化较快，因此可以用它来构造平滑的学习率衰减过程。

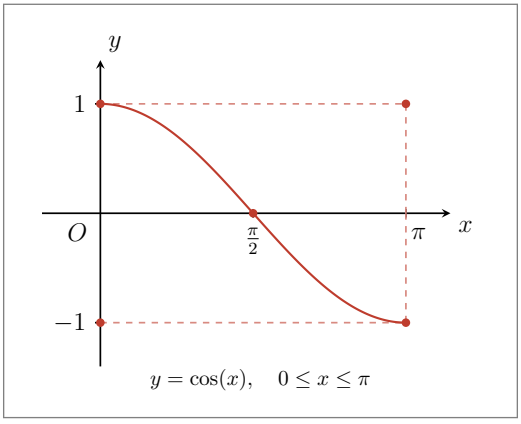


图 3.6 余弦函数 $y = \cos(x)$ 在区间 $[0, \pi]$ 上的图像。曲线自 $(0, 1)$ 单调递减，于 $x = \pi/2$ 处过零，最终到达 $(\pi, -1)$，且两端斜率为零。

这就是**余弦退火（Cosine Annealing）**的基本思想，其学习率计算公式为：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{T_{\text{cur}}}{T_{\max}}\pi\right)\right)$$

其中，$\eta_{\max}$ 和 $\eta_{\min}$ 分别表示学习率的上限和下限，T_{cur} 表示当前的训练轮次，$T_{\max}$ 表示总

的训练轮次。随着训练进行，T_{cur} 从 0 逐渐增加到 $T_{\max}$，余弦函数的自变量也会从 0 逐渐变化到 π，因此学习率会按照余弦曲线的变化规律，从 $\eta_{\max}$ 连续、平滑地下降到 $\eta_{\min}$。

为了更直观地理解这个过程，我们代入一组具体的参数。假设设置 $\eta_{\max}=0.1$、$\eta_{\min}=0$，总训练轮次 $T_{\max}=100$ 个 Epoch，那么公式可以简化为：

$$\eta_t = 0.05\left(1+\cos\left(\frac{T_{\text{cur}}}{100}\pi\right)\right)$$

在训练刚开始时，$T_{\text{cur}}=0$，此时 $\cos(0)=1$，因此 $\eta_0=0.1$，学习率处于最大值。训练进行到四分之一，即 $T_{\text{cur}}=25$ 时，有 $\cos(\pi/4)\approx 0.707$，此时学习率约为 0.085，相比初始值仅下降了 0.015，说明训练初期下降得比较缓慢。而当训练进行到一半，即 $T_{\text{cur}}=50$ 时，$\cos(\pi/2)=0$，学习率下降到 0.05。继续训练到四分之三，即 $T_{\text{cur}}=75$ 时，$\cos(3\pi/4)\approx -0.707$，学习率约为 0.015，此时下降速度明显加快。最后，当训练进行到 $T_{\text{cur}}=100$ 时，$\cos(\pi)=-1$，学习率恰好下降到 $\eta_{\min}=0$。

由这个例子可以看出，余弦退火不会让学习率突然下降，而是让它在训练初期下降较慢，中期较快，后期再次放缓。相比阶跃衰减，这种变化更加连续、平滑，使参数更新能够从前期的快速调整逐渐过渡到后期的精细微调。

学习率预热（Warmup）

在使用余弦退火或设置较大的初始学习率时，通常还会搭配一种常见的技术：**学习率预热（Warmup）**。在训练刚开始时，模型的参数通常是随机初始化的，梯度也可能存在较大的波动。如果一开始就直接使用较大的学习率 $\eta_{\max}$，参数更新幅度可能过大，使训练初期的 Loss 出现明显震荡，甚至导致训练不稳定。

因此，学习率预热的核心思想，就是在训练初期的几个 Epoch 内，不直接使用较大的学习率，而是让学习率从一个较小的值逐渐增加到设定的最大值 $\eta_{\max}$。例如，可以从 0 开始，按照线性方式逐步增加学习率。完成预热后，再进入余弦退火阶段，使学习率从 $\eta_{\max}$ 逐渐下降到 $\eta_{\min}$。

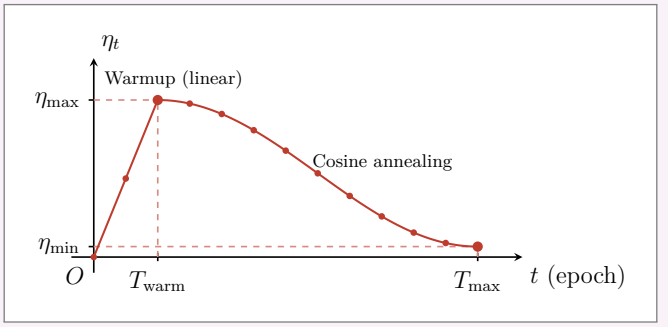


图 3.7 学习率预热和余弦退火的学习率调度示意图。前 T_{warm} 个 epoch 内学习率从 0 线性升至 $\eta_{\max}$，随后按余弦曲线平滑衰减至 $\eta_{\min}$，两段在 $t=T_{\text{warm}}$ 处连续衔接。

如图 3.7所示，整个训练过程可以分为两个阶段：首先通过学习率预热逐渐增大学习率，使模型平稳进入训练状态；随后通过余弦退火逐步减小学习率，使参数更新从快速调整过渡到精细微调。因此，学习率预热 + 余弦退火是一种常见的学习率调度组合。

3.2 批量策略

我们接着看梯度下降的参数更新公式：

$$\theta \leftarrow \theta - \eta \cdot \frac{\partial \mathcal{L}}{\partial \theta}$$

在之前的推导中，我们实际上默认了一种理想情况：每次更新参数时，模型都会使用整个训练数据集来计算平均损失，再根据得到的梯度更新参数。这样得到的梯度能够充分利用所有训练样本，因此方向最为准确。

但在实际训练中，这种做法并不总是可行。假设一个数据集包含 100 万张图片，如果每更新一次参数，都需要将 100 万张图片全部输入网络，完成一次前向传播和反向传播，那么单次参数更新的计算成本将不可接受。于是，一个问题就出现了：每次更新参数时，到底应该使用多少个样本来计算梯度?

最直接的做法，就是使用全部训练数据。如果数据集规模较小，能够一次性装入内存，那么让所有样本共同参与一次参数更新，可以得到最完整的梯度信息。这就是**全批量梯度下降 (Batch Gradient Descent, BGD)**，其梯度计算公式为：

$$\frac{\partial \mathcal{J}}{\partial \theta} = \frac{1}{n} \sum_{i=1}^{n} \frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$$

其中，n 表示训练数据集中的样本总数。由于每次更新都使用全部样本，BGD 得到的梯度估计最充分，参数更新过程也更加平稳。但与此同时，每进行一次参数更新都必须遍历整个数据集，因此数据规模越大，单次更新的计算成本就越高。

既然使用全部数据计算梯度的代价太高，那么我们也可以反过来，只使用一个样本进行参数更新。这就是**随机梯度下降 (Stochastic Gradient Descent, SGD)**。每次随机选取一个样本每次随机选取一个样本 i，直接使用该样本的梯度代表整个数据集的梯度：

$$\frac{\partial \mathcal{J}}{\partial \theta} \approx \frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$$

这样一来，每次参数更新只需要处理一个样本，计算成本大幅降低，参数也能够更加频繁地更新。但由于单个样本只能反映整个数据集的一小部分信息，梯度估计的方差较大，参数更新会受到明显的随机波动影响。因此，SGD 的优化轨迹通常比较曲折，在最优解附近也容易出现震荡。但是，这种随机性有时反而能够帮助模型跳出某些局部区域，使参数继续寻找更好的解。

全批量梯度下降计算成本太高，而随机梯度下降又容易产生较大的波动，我们可以采取一个折中方案：每次随机选取一小批样本，用这一批样本的平均梯度来表示整个数据集的梯度。这就是**小批量梯度下降 (Mini-batch Gradient Descent)**，也是现代深度学习中最常用的训练方式。其梯度计算公式为：

$$\frac{\partial \mathcal{J}}{\partial \theta} \approx \frac{1}{b} \sum_{i=1}^{b} \frac{\partial \mathcal{L}^{(i)}}{\partial \theta}$$

其中，b 表示每批抽取的样本数量，即 Batch Size。Mini-batch 的核心优势在于，它在计算效率和训练稳定性之间取得了较好的平衡。一方面，每次只使用一小部分样本计算梯度，不需

要像全批量梯度下降那样遍历整个数据集，因此能够显著降低单次参数更新的计算成本；另一方面，一个 Batch 中包含多个样本，对它们的梯度进行平均后，又能够减小单个样本带来的随机波动，使参数更新更加稳定。因此，在现代深度学习中，通常采用 Mini-batch 的方式进行训练。并且，一般来说，当我们谈论“梯度下降”时，指的就是这种基于 Mini-batch 的梯度下降方法。

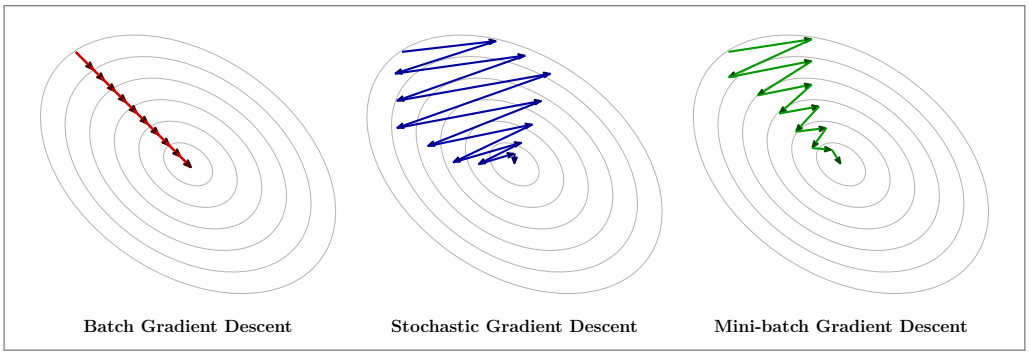


图 3.8 BGD、SGD 与 Mini-batch 三种梯度下降轨迹在损失等高线图上的对比图。等高线由外向内递减、中心为极小值点。

如图 3.8所示，我们以损失函数的等高线图作为背景，等高线由外向内逐渐降低，中心位置为极小值点。可以看到，BGD 使用全部样本计算梯度，更新方向较为稳定，因此沿着山谷平滑地向谷底逼近，轨迹近乎直线；SGD 每次只使用一个样本，梯度受到单个样本的影响较大，因此更新过程中会出现明显的随机波动，轨迹也更加曲折；Mini-batch 则介于两者之间，虽然仍然存在一定的抖动，但整体更新方向更加稳定，既保留了一定的随机性，又能够较快地向最优解靠近。

Epoch、Batch Size 和 Iteration

在深度学习的训练过程中，Epoch、Batch Size 和 Iteration 是三个非常重要的基本概念。其中，一个 **Epoch（轮次）**表示模型将整个训练数据集完整地训练一遍，也就是完成一轮训练。由于模型通常需要反复学习训练数据，才能逐渐调整出合适的参数，因此一次完整的模型训练通常需要进行多个 Epoch。

Batch Size（批大小）表示每次参数更新时，模型同时处理的样本数量，也就是前面 Mini-batch 中的 b。例如，当 Batch Size 设置为 32 时，模型每次使用 32 个样本计算梯度，并根据计算得到的梯度更新一次模型参数。Batch Size 越小，每次更新使用的样本越少，梯度波动通常越明显；Batch Size 越大，梯度估计通常更加稳定，但需要更多的计算资源和显存。

在明确了 Epoch 和 Batch Size 之后，**Iteration（迭代次数）**也就比较容易理解了。一次 Iteration 就是模型完成一次参数更新。因此，在一个 Epoch 中，模型会按照 Batch Size 将整个训练数据集分成若干个 Batch，每处理完一个 Batch，就进行一次参数更新，也就完成一次 Iteration。

例如，假设训练数据集包含 1000 个样本，Batch Size 设置为 32，则完成一个 Epoch

需要：

$$\left\lceil \frac{1000}{32} \right\rceil = 32$$

次 Iteration。由于 $32 \times 31 = 992$，因此前 31 个 Batch 各包含 32 个样本，最后一个 Batch 包含剩余的 8 个样本。如果总共训练 50 个 Epoch，那么整个训练过程就需要进行：

$$32 \times 50 = 1600$$

次参数更新，也就是 1600 次 Iteration。

因此，可以简单地记住：Epoch 表示完整遍历数据集的次数，Batch Size 表示一次使用多少个样本，Iteration 表示进行多少次参数更新。

需要说明的是，严格来说，随机梯度下降（SGD）是指每次使用单个样本计算梯度并更新参数。但在现代深度学习中，实际训练通常以 Mini-batch 为单位进行参数更新。例如，`PyTorch` 中的 `torch.optim.SGD` 虽然名称仍然使用 SGD，但实际训练时也是在每个 Mini-batch 上计算梯度并更新参数。因此，在后面的内容中，我们提到 SGD 时，通常指的就是这种基于 Mini-batch 的随机梯度下降。

3.3 动量法

SGD（此处及后文的 SGD 均指 Mini-batch GD，下同）能够在计算效率和训练稳定性之间取得较好的平衡。但在实际训练中，损失函数的形状往往并不规则。如图 3.9所示，有些损失函数会呈现出极其狭长的峡谷。

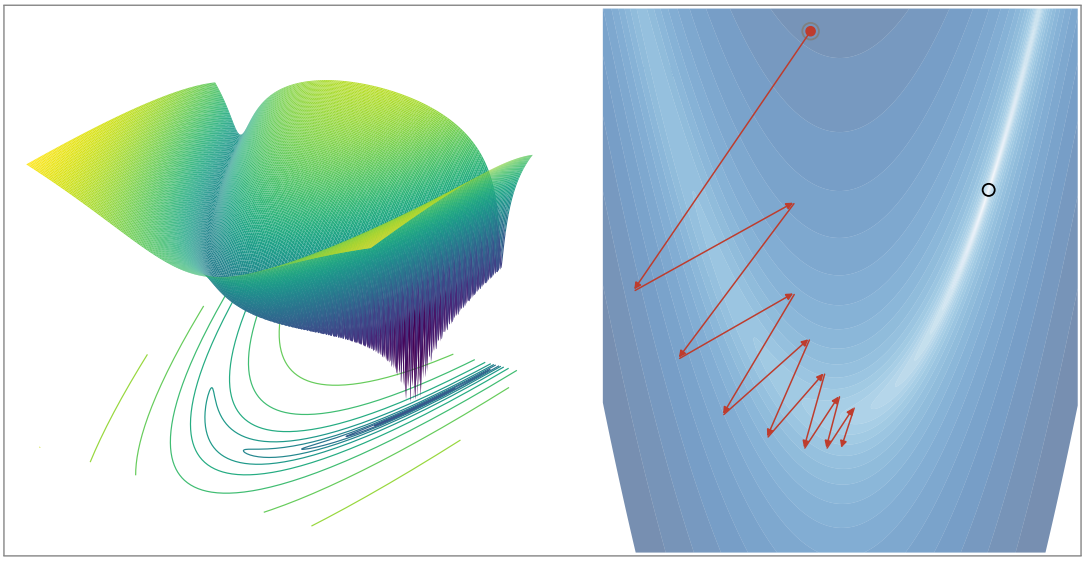


图 3.9 病态曲率损失面（左）及其上的 SGD 参数优化轨迹（右）。左图等高线围成狭窄且弯曲的谷，垂直谷方向的曲率与梯度远大于沿谷方向；右图中 SGD 的更新步因此在谷壁之间来回横跳，难以沿谷方向向极小值点（白色圆圈）有效推进。

这种地形被称为**病态曲率（Ill-conditioned Curvature）**。在这种地形中，垂直于峡谷底部的方向上，损失函数变化很快，梯度往往较大；而沿着峡谷底部的方向，损失函数变化较慢，梯度相对较小。如果使用普通的 Mini-batch SGD 进行优化，参数更新就容易受到大梯度方向的影响，在峡谷两侧反复来回移动，而真正指向极小值的方向更新却比较缓慢。最终，参数会沿着一条不断左右摆动的“之”字形路径向前移动，虽然 Loss 在逐渐下降，但收敛速度却非常缓慢。

为什么会出现这种现象？一个重要原因是，普通 SGD 在每次更新参数时，只根据当前时刻的梯度决定更新方向，而不会保留之前的更新信息。在狭长的峡谷中，垂直于峡谷底部的方向梯度较大，因此参数很容易被推向峡谷另一侧；一旦越过谷底，梯度方向又会发生改变，于是下一次更新又会被推向另一侧。这样反复进行，参数就会在峡谷两侧不断来回摆动，而沿着峡谷底部方向的梯度较小，参数向最优解前进的速度反而很慢。那么，我们能不能让参数在更新时保留之前的运动趋势，而不是每次都只根据当前梯度决定方向？也就是让参数在考虑当前梯度的同时，也参考过去的更新方向，从而减少这种反复摆动。

那为什么考虑之前的更新方向，就能够减小这种摆动呢？这是因为在狭长峡谷中，参数在垂直于峡谷底部的方向上会不断受到方向相反的梯度影响：这一步被推向左侧，下一步又被推向右侧。如果把之前的更新方向保留下来，这些相反方向的更新会在累积过程中相互抵消，从而减弱参数在横向上的来回摆动。与此同时，沿着峡谷底部方向的梯度通常具有较强的一致性，连续多次更新会在相同方向上逐渐积累，使参数能够更快地沿着峡谷底部向最优解移动。

根据这个思路，我们可以在参数更新过程中引入一个速度变量 v_t，用它记录历史梯度积累形成的运动趋势。这样，参数的更新就不再只取决于当前梯度，而是会同时受到之前更新方向的影响。这种利用历史梯度信息来改进参数更新的方法，就是**动量法（Momentum）**。

有了速度变量之后，我们就可以将这一思想写成数学形式：

$$v_t = \gamma v_{t-1} + \eta g_t$$

$$\theta_t = \theta_{t-1} - v_t$$

其中，v_{t-1} 表示上一时刻累积的速度，γ 表示动量系数，用来控制历史信息保留的程度，通常取 0.9；η 表示学习率，g_t 表示当前时刻的梯度，即：

$$g_t = \nabla \mathcal{L}(\theta_{t-1})$$

从这个公式可以看出，当前的速度 v_t 由两部分组成：一部分来自上一时刻已经积累的速度，另一部分来自当前梯度带来的更新。也就是说，参数这一步往哪里走，不再完全由当前梯度决定，而是会在历史运动趋势的基础上进行调整。

进一步展开这个递推关系，可以得到：

$$v_t = \eta g_t + \gamma \eta g_{t-1} + \gamma^2 \eta g_{t-2} + \cdots$$

可以看到，越近的历史梯度影响越大，越早的梯度影响则会按照 γ 的幂逐渐减弱。因此，动量法本质上是在不断积累过去的梯度信息，同时让较久远的信息逐渐衰减。这也使得参数更

新具有了一定的“惯性”。

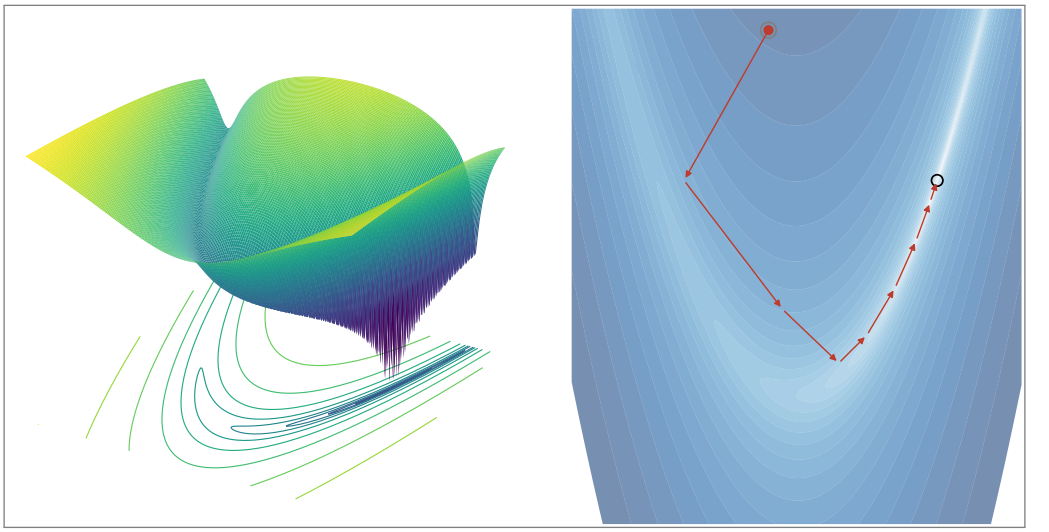


图 3.10 病态曲率损失面（左）与使用动量法的参数优化轨迹（右）。在窄而弯的谷地形上，动量法通过速度项累积历史梯度，抵消横向震荡并叠加沿谷推进；轨迹仅在初期轻微过冲，随后即沿谷底平滑收敛至极小值点（白色圆圈）。

使用动量法之后的优化结果如图 3.10所示。将它与普通 SGD 的优化轨迹进行对比可以发现，普通 SGD 容易在峡谷两侧反复震荡，形成明显的“之”字形路径；而加入动量后，历史梯度带来的累积效应能够减弱横向摆动，同时增强沿峡谷方向的运动，使参数更加平稳、快速地向极小值靠近。由此，动量法不仅能够缓解病态曲率带来的震荡问题，还能够利用历史信息加快收敛。

3.4 自适应学习率算法

我们通过学习率衰减让学习率随着训练过程逐渐减小，通过动量法利用历史梯度改善参数更新方向。不过，这些方法都有一个共同点：在同一个时刻，网络中的所有参数都使用相同的学习率。虽然不同参数对应的梯度大小不同，因此实际的参数更新幅度也会有所不同，但学习率本身对所有参数都是统一的。然而，在实际任务中，不同参数的梯度情况可能存在很大差异。有些参数长期处于较大的梯度区域，需要逐渐减小更新步长；而另一些参数的梯度一直较小，则可能需要更大的步长才能得到充分更新。

那么，我们能不能让不同参数根据自身的历史梯度，自动调整各自的学习率？这就是**自适应学习率算法**的基本思想。与统一设置一个全局学习率不同，它会根据每个参数过去的梯度情况，动态调整这个参数当前的有效学习率。

3.4.1 AdaGrad

最直接的做法是：既然不同参数的更新情况不同，那么就分别记录每个参数过去的梯度。如果某个参数长期收到较大的梯度，说明它已经被频繁地调整过，那么后续可以逐渐减小它

的学习率；反过来，如果某个参数历史上的梯度一直较小，那么就保留较大的学习率，让它能够继续得到充分更新。那么，怎么记录一个参数过去的梯度呢？我们可以把从训练开始到当前时刻为止的梯度平方逐步累加起来，并用这个累积结果调整当前的学习率。这就是 **AdaGrad（Adaptive Gradient）** 的基本思想[1]。其更新形式可以写为：

$$G_t = \sum_{k=1}^{t} g_k^2$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

其中，g_t 表示当前时刻的梯度，η 表示基础学习率，G_t 表示从第 1 步到第 t 步的梯度平方累积和，ϵ 是一个很小的常数（如 10^{-8}），用于避免分母为 0。

从公式中可以直接看出，G_t 越大，分母就越大，当前参数的有效学习率也就越小。因此，那些历史梯度较大、更新较频繁的参数，会逐渐减小更新步长；而历史梯度较小的参数，则能够保留相对较大的学习率。

我们可以用一个简单的例子来理解这一点。假设模型中有两个参数 w_1 和 w_2，虽然它们都会在每次参数更新时参与计算，但由于不同参数对 Loss 的影响不同，它们在不同训练步骤中得到的梯度大小也可能不同。假设基础学习率 $\eta = 1$。对于参数 w_1，每一步的梯度都约为 1；而对于参数 w_2，每一步的梯度都约为 0.1。训练进行到第 100 步时，两个参数的梯度平方累积和分别为：

$$G_{100}^{(1)} = 100 \times 1^2 = 100$$

$$G_{100}^{(2)} = 100 \times 0.1^2 = 1$$

因此，AdaGrad 为两个参数计算出的有效学习率分别为：

$$\eta_{100}^{(1)} = \frac{1}{\sqrt{100}} = 0.1$$

$$\eta_{100}^{(2)} = \frac{1}{\sqrt{1}} = 1$$

可以看到，虽然两个参数使用的是相同的基础学习率 $\eta = 1$，但由于它们历史上的梯度大小不同，AdaGrad 会自动为它们分配不同的有效学习率：梯度长期较大的 w_1 使用较小的学习率，而梯度较小的 w_2 则保留较大的学习率。这就是 AdaGrad 为不同参数自适应调整学习率的核心思想。

不过，AdaGrad 也存在一个明显的问题。G_t 是一个不断累加的量，只会增加而不会减少。随着训练进行，越来越多的历史梯度被加入其中，分母会不断变大，导致有效学习率持续下降。训练进行到后期时，即使当前梯度仍然比较明显，学习率也可能已经小到几乎无法继续更新参数，最终导致训练停滞。

[1] AdaGrad 由 John Duchi、Elad Hazan 和 Yoram Singer 于 2011 年提出，相关工作发表于论文*Adaptive Subgradient Methods for Online Learning and Stochastic Optimization*。

3.4.2 RMSProp

面对 AdaGrad 的这一问题，一个比较直接的改进思路就是：没有必要永久保留所有历史梯度，只需要重点保留最近的梯度信息，让较早的梯度逐渐失去影响。

实际上，我们在前面的动量法中已经有过类似的思路：

$$v_t = \eta g_t + \gamma \eta g_{t-1} + \gamma^2 \eta g_{t-2} + \cdots$$

可以看到，越新的梯度对当前更新的影响越大，而较早的梯度会随着时间推移逐渐减弱。现在，我们可以采用类似的方式来记录梯度平方：不再把所有历史梯度平方简单地累加起来，而是给当前梯度平方分配一定的权重，同时保留过去的信息，并让更久远的信息逐渐减弱。这样得到的量会更加关注近期的梯度变化，也不会随着训练时间的增加而无限增大。

这就是 **RMSProp (Root Mean Square Propagation)** 的基本思想[2]。它不再像 AdaGrad 那样将所有历史梯度平方不断累加，而是对近期的梯度平方进行加权平均：

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1-\gamma) g_t^2$$

其中，g_t 表示第 t 次参数更新时计算得到的梯度，g_t^2 表示当前梯度的平方；$E[g^2]_t$ 表示截至当前时刻，对近期梯度平方进行加权后得到的结果，可以理解为对参数近期梯度大小的一种记录。$E[g^2]_{t-1}$ 则是上一步的记录，当新的梯度平方 g_t^2 到来后，RMSProp 会将旧的记录和当前的梯度平方按照不同的权重进行组合。其中，γ 表示对历史信息的保留程度，通常取 0.9。例如，当 $\gamma = 0.9$ 时，上一步的记录保留 90%，当前梯度平方贡献 10%。随着这一过程不断重复，较早的梯度会被一次又一次地乘以 γ，因此其影响会逐渐减弱，而较新的梯度则能够更快地反映到当前的记录中。

我们进一步看一下 $\gamma = 0.9$ 时，这种历史信息会衰减得有多快。一个较早的梯度经过多次更新后，其影响会不断乘以 0.9。经过 10 步后，

$$0.9^{10} \approx 0.35$$

也就是说，它原本的影响只剩下约 35%；经过 20 步后，

$$0.9^{20} \approx 0.12$$

只剩下约 12%；到了 30 步后，

$$0.9^{30} \approx 0.04$$

影响已经降到约 4%。因此，$\gamma = 0.9$ 会让较近的历史信息保留下来，而较久远的信息则逐渐被淡化。

得到 $E[g^2]_t$ 之后，我们再利用它来调整当前梯度 g_t 的有效学习率：

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

[2] RMSProp 并未以正式论文的形式发表，其提出与相关说明主要见 Tijmen Tieleman 和 Geoffrey Hinton 于 2012 年发布的课程讲义*RMSprop: Divide the Gradient by a Running Average of its Recent Magnitude*（见课件第 26–31 页；虽封面署名其他助教，但第 29 页注明除以梯度均方根的核心技巧源自 Tijmen Tieleman，故学术界通常将其列为第一作者引用），课程名称为 *Neural Networks for Machine Learning*。

可以看到，$E[g^2]_t$ 越大，分母就越大，当前梯度对应的有效学习率也就越小；反之，$E[g^2]_t$ 越小，学习率就越大。因此，RMSProp 会根据参数近期的梯度大小，自动调整它的更新步长。

这样一来，AdaGrad 中“只记不忘”的累积方式就变成了“边记边忘”。由于 $E[g^2]_t$ 采用的是近期梯度平方的加权平均，而不是像 G_t 那样不断累加，因此不会随着训练进行而无限增大，从而避免学习率不断衰减到接近于 0，使模型在训练后期仍然能够保持有效的参数更新。不过，RMSProp 解决的主要是每个参数应该走多远的问题，而没有解决应该往哪个方向走的问题。它通过历史梯度平方来调整参数的更新步长，但更新方向仍然主要由当前梯度 g_t 决定。因此，如果当前 Batch 带来的梯度存在较大的随机波时，参数的更新方向也可能随之发生变化，仍然容易受到当前梯度噪声的影响。

3.4.3 Adam

既然动量法利用历史梯度的信息，解决了参数往哪个方向走的问题；RMSProp 则根据梯度大小调整参数的更新步长，解决了参数走多远的问题。那我们能不能把这两种方法结合起来？既利用历史梯度来平滑更新方向，又根据每个参数自身的梯度情况自动调整学习率。这就是 **Adam（Adaptive Moment Estimation）** 的基本思想[3]。

Adam 同时维护两个量：一阶矩 m_t 和二阶矩 v_t[4]。一阶矩记录梯度的加权平均，用于平滑更新方向；二阶矩记录梯度平方的加权平均，用于调整参数的有效学习率。其核心公式为：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

其中，β_1 控制历史梯度信息的保留程度，β_2 控制历史梯度平方信息的保留程度。可以看到，第一个公式与动量法的思想类似，通过保留历史梯度来平滑当前的更新方向；第二个公式与 RMSProp 类似，通过记录近期梯度平方的大小来调整更新步长。

不过，m_t 和 v_t 在训练开始时通常都初始化为 0。这会带来一个问题：在最初几次更新中，它们积累的梯度信息还比较少，因此得到的数值会明显偏小，不能很好地反映当前梯度的真实情况。例如，假设只有第 1 次更新时产生了梯度，之后暂时不考虑新的梯度，那么由于 $m_0 = 0$，第一次计算得到的 m_1 只保留了当前梯度的一部分，而不是完整的梯度信息。随着训练继续进行，这种从 0 开始累积带来的低估会逐渐减弱。为了解决这个问题，Adam 引入了**偏差修正（Bias Correction）**：

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

这里的 $1-\beta_1^t$ 和 $1-\beta_2^t$ 就是用来补偿初始化为 0 所造成的低估。训练刚开始时，β_1^t 和 β_2^t 还比较接近 1，分母较小，因此修正幅度较大；随着 t 不断增加，β_1^t 和 β_2^t 逐渐接近 0，修正作用也就越来越小。

---

[3] Adam 由 Diederik Kingma 和 Jimmy Ba 于 2015 年提出，相关工作发表于论文*Adam: A Method for Stochastic Optimization*。

[4] 一阶矩和二阶矩来自统计学中的**矩（Moment）**：一阶原点矩为 $E[X]$，二阶原点矩为 $E[X^2]$。在 Adam 中，m_t 是对梯度一阶矩的指数加权估计，用于反映梯度的平均方向；v_t 是对梯度二阶矩的指数加权估计，用于反映梯度的尺度。需要注意的是，v_t 对应的是 $E[g_t^2]$，而不是方差。

得到修正后的 $\hat{m}_t$ 和 $\hat{v}_t$ 后，Adam 最终的参数更新公式为：

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

整个过程可以理解为：首先根据当前梯度 g_t 更新一阶矩和二阶矩；然后对这两个统计量进行偏差修正；最后利用修正后的一阶矩确定参数更新方向，再利用二阶矩调整更新步长。于是，Adam 就把“方向”和“步长”两个问题统一到了同一个更新公式中。

为了更直观地理解 Adam 的作用，我们还是回到前面的峡谷问题。假设某个参数在优化过程中不断来回摆动，得到的梯度依次为：

$$g_1 = 1, \qquad g_2 = -1, \qquad g_3 = 1, \qquad g_4 = -1$$

可以看到，梯度方向不断发生变化。此时，一阶矩会对这些历史梯度进行加权平均，正负交替的梯度会在累积过程中相互抵消，因此一阶矩的变化会比原始梯度平缓得多，从而减小单次梯度波动对更新方向的影响。

而与此同时，这些梯度的绝对值始终为 1，说明这个参数一直处于梯度较大的区域。因此，二阶矩会持续记录较大的梯度平方，并通过分母 $\sqrt{\hat{v}_t}$ 压缩有效学习率，从而避免参数在梯度较大的区域进行过大的更新。

所以，Adam 实际上同时解决了两个问题：一阶矩负责往哪个方向走，二阶矩负责走多远。一阶矩继承了动量法的思想，二阶矩则继承了 RMSProp 的思想，而偏差修正则进一步改善了训练初期的统计估计。

从整个优化算法的发展过程来看，可以看到一条清晰的演进路线：动量法利用历史梯度平滑更新方向，减少参数震荡；AdaGrad 进一步让不同参数拥有不同的有效学习率，但由于历史梯度平方不断累积，学习率可能逐渐衰减过小；RMSProp 让久远的历史信息逐渐衰减，解决了 AdaGrad 学习率持续下降的问题；最终，Adam 将动量法和 RMSProp 的思想结合起来，同时对更新方向和步长进行调整。不过，Adam 并非在所有任务中都一定优于其他优化器，因此实际训练时仍需要根据具体任务选择合适的方法。

> 我们已经掌握了多种优化算法，能够让模型更加高效、稳定地更新参数。那么，在实际训练深度神经网络时，还会遇到其他问题吗？